

АНОТАЦІЯ

Бакалаврська кваліфікаційна робота виконана студентом групи ІІІ-42 Ковалем Денисом Петровичем. Тема «Система для автоматичної нормалізації суржику та діалектних форм сучасної української мови». Робота направлена на здобуття ступеня бакалавра за спеціальністю 122 «Комп'ютерні науки».

Об'єктом дослідження є процеси автоматичної нормалізації нелітературних форм сучасної української мови - суржику та регіональних діалектів.

Предметом дослідження є методи формування паралельних корпусів для малоресурсних мовних варіантів і методи навчання нейронних моделей нормалізації діалектного і суржикового тексту.

У роботі зібрано паралельний корпус із 125 615 навчальних, 13 756 валідаційних і 200 тестових прикладів для чотирьох різновидів - гуцульського, бойківського, закарпатського діалектів і суржику. Для гуцульського сформовано 9 853 пари ручної анотації, решту представлено переважно синтетикою, згенерованою правилами методами (pymorphy2) і великою мовною моделлю Mistral Small 24B. Тестову вибірку зібрано виключно вручну.

Навчено дві моделі: кодер-декодерну umt5-base (повне донавчання) і декодерну MamayLM на базі Gemma-3-4B (донавчання QLoRA). Якість оцінено за метриками BLEU, chrF++ і TER у порівнянні з нуль-шотовими моделями GPT-4o, Gemini 2.0 Flash і Mistral Large.

Навчені моделі перевершили нуль-шотові великі мовні моделі на різних видах із якісними даними: chrF++ становить 93,58 на суржику і 86,90 на гуцульському діалекті (umt5-base). Для бойківського і закарпатського перевага залишилася за комерційними моделями, що підтверджує роль ручних анотацій. Практичним результатом є вебзастосунок «Лексикон» (NiceGUI, FastAPI) з голосовим введенням на основі umt5-base і SpeechBrain.

Обсяг роботи - пояснювальна записка - 76 с., 19 рис., 8 табл.

Ключові слова: нормалізація тексту, суржик, діалекти, нейронний машинний переклад, велика мовна модель, донавчання.

ABSTRACT

The bachelor's qualification work was completed by Koval Denys Petrovych, a student of group AI-42. The topic is "System for Automatic Normalization of Surzhyk and Dialectal Forms of the Modern Ukrainian Language." The work is aimed at obtaining a bachelor's degree in specialty 122 "Computer Science."

The object of research is the process of automatic normalization of non-standard forms of modern Ukrainian - surzhyk and regional dialects - to literary standard Ukrainian.

The subject of research is methods for constructing parallel corpora for low-resource language varieties, and methods for training neural models for the task of text normalization.

A parallel corpus of 125,615 training examples was assembled for four language varieties: Hutsul, Boikivian, and Transcarpathian dialects, as well as surzhyk. Synthetic data was generated using rule-based approaches (SurzhykErrorifier) and the Mistral-Small-24B language model. Two neural models were trained: an encoder-decoder model (umt5-base, full fine-tuning, 40 epochs) and a decoder-only large language model (MamayLM based on Gemma-3-4B, QLoRA parameter-efficient fine-tuning, 3 epochs). Quality was evaluated using BLEU, chrF++, and TER metrics against zero-shot large language model baselines: GPT-4o, Gemini 2.0 Flash, and Mistral Large. The fine-tuned umt5-base model achieved chrF++ = 93.58 on surzhyk and chrF++ = 86.90 on the Hutsul dialect. A web application "Leksykon" was implemented with voice input support via the SpeechBrain ASR module.

Keywords: text normalization, surzhyk, Ukrainian dialects, large language model, QLoRA, parallel corpus, Hutsul dialect, Boikivian dialect, Transcarpathian dialect.

1. Introduction and Problem Statement

The Ukrainian language occupies a complex sociolinguistic position in modern Ukraine. Standard literary Ukrainian coexists with a wide spectrum of non-standard varieties, ranging from regionally grounded dialects to urban contact languages. Two categories of non-standard Ukrainian speech are of particular relevance for natural language processing: *surzhyk* and the western Ukrainian regional dialects.

Surzhyk is a sociolinguistic phenomenon that emerged during the Soviet era as a result of intensive contact between Ukrainian and Russian. Unlike geographic dialects, surzhyk is not bound to a specific region: it is widely spoken in central and eastern Ukraine, primarily in urban areas. Linguistically, surzhyk is characterized by lexical and morphological mixing without a stable phonological system. Common manifestations include Russian lexical substitutions (Ukrainian *але* “but” replaced by Russian *но*; *зупинка* “bus stop” by *остановка*), Russian infinitive endings (-ть instead of -ти), and calqued conjunctions (*якщо* “if” replaced by *если*).

The Carpathian and western Ukrainian region is home to three dialects addressed in this work. The *Hutsul dialect* is spoken in the Carpathian mountain communities and is relatively well documented. Its distinctive features include front-vowel stress shifts (*шанка* → *шенка*), the reflexive particle *си* (instead of standard *ся*), and archaic verbal suffixes (-*єм* for third-person singular). The *Boikivian dialect*, spoken in the foothills of the Carpathians, is more archaic and phonologically conservative. Its characteristic features include the preservation of an unrounded back vowel [ы] distinct from standard [и] (*бик* → *бык*) and the shift of the lateral [л] to the bilabial approximant [ʏ] in coda position (*галка* → *гавка*). The *Transcarpathian dialect* is the most distant from the literary norm, shaped by centuries of contact with Hungarian, Slovak, and Romanian. Key features include the vowel shift *o* → *u* in closed syllables (*кінь* → *кунь*), preservation of the archaic [ы], and numerous contact borrowings.

Automatic text normalization - the task of mapping a non-standard input sequence *x* to literary standard output *y* - occupies a position between grammatical

error correction (GEC) and machine translation (MT). Unlike GEC, dialect normalization targets systematic linguistic features that are not “errors” from the speaker’s perspective, but rather reflect a different language variety. Unlike MT, normalization operates within a single language family, with significant lexical overlap between input and output.

The lack of specialized resources for Ukrainian dialect normalization is a critical gap. Public corpora for Ukrainian grammatical error correction - UA-GEC and Spivavtor - are oriented toward standard Ukrainian texts and do not cover dialectal input. No public parallel corpus exists for Boikivian or Transcarpathian dialects. Existing work on Ukrainian dialectal NLP is sparse: Vuyko Mistral targets Hutsul-to-standard translation using limited in-house data; Bytes to Borsch focuses on adapting large language models to standard Ukrainian rather than dialectal processing. No prior work has addressed all four varieties simultaneously within a unified framework.

The present work addresses this gap through four contributions: (1) construction of a parallel normalization corpus of 139,576 sentence pairs for four non-standard Ukrainian varieties, combining manual annotation, rule-based generation, and large language model synthesis; (2) training of two neural normalization models - an encoder-decoder (umt5-base) and a decoder-only model (MamayLM, Gemma-3-4B with QLoRA) - on this corpus; (3) comparative evaluation against zero-shot commercial large language model baselines (GPT-4o, Gemini 2.0 Flash, Mistral Large); and (4) deployment of the “Leksykon” web application integrating normalization with automatic speech recognition.

2. Linguistic Characteristics of the Four Varieties

The four non-standard varieties addressed in this work differ substantially in their linguistic distance from the literary norm and in the availability of annotated resources. The linguistic distance ordering - surzhyk, Hutsul, Boikivian, Transcarpathian (from least to most distant) - directly predicts normalization difficulty, as confirmed by the evaluation results in Section 5.

Surzhyk ($\approx 50,523$ training pairs) is characterized by Russian lexical and morphological mixing without a stable phonological system. Normalization reduces largely to lexical and morphological substitution: a sufficient dictionary of Russian-Ukrainian lexical correspondences and morphological transformation rules captures a large fraction of surzhyk utterances. This means that rule-based and LLM-synthetic training data closely resembles real surzhyk usage and yields strong training signal.

Hutsul dialect ($\approx 36,640$ training pairs) is spoken in the Carpathian mountain communities and is characterized by front-vowel stress shifts, the reflexive particle *cu* instead of standard *ся*, and archaic verbal suffixes. The availability of 9,853 manually annotated parallel pairs provides clean training signal for the phonological patterns that rules alone cannot capture reliably. These pairs were sourced from a public dataset and validated by native speakers.

Boikivian dialect ($\approx 29,754$ training pairs) is spoken in the Carpathian foothills and is phonologically archaic: it preserves an unrounded back vowel [ɤ] distinct from standard [ɯ], and shifts [ɲ] to bilabial [ɣ] in coda position. No manually annotated corpora are publicly available; training data was generated entirely synthetically, introducing a distribution mismatch that limits model performance.

Transcarpathian dialect ($\approx 29,605$ training pairs) is the most distant from the literary norm, shaped by centuries of contact with Hungarian, Slovak, and Romanian. Key features include the vowel shift $o \rightarrow u$ in closed syllables, preservation of archaic [ɤ], and numerous contact borrowings with no Ukrainian equivalents. Training data is entirely synthetic, and the distribution mismatch is most severe here.

3. Corpus Construction

The corpus was constructed using a “standard-to-dialect” strategy: given a sentence in standard literary Ukrainian, the system generates the corresponding dialectal or surzhyk variant. This approach guarantees correct targets by construction - the standard-side output comes from quality source texts rather than potentially incorrect normalization outputs. At inference time, the direction is inverted (dialect $\rightarrow$ standard), and the model learns to normalize by reversing the transformation

patterns seen during training.

The primary source of standard Ukrainian sentences was the Spivavtor corpus, a collection of Ukrainian text-editing pairs that provides high-quality literary Ukrainian prose. For the Hutsul dialect, 9,853 manually annotated parallel pairs (dialect $\rightarrow$ standard) from a public dataset were incorporated directly and inverted for training.

Surzhyk data was generated using the SurzhykErrorifier class, which implements two-phase generation with a morphological analyzer (pymorphy2) and a manually compiled surzhyk lexicon. Phase 1 applies dictionary-based phrase substitution (longest-first matching to avoid partial replacements). Phase 2 applies morphological substitution: the correct Ukrainian word form is identified via pymorphy2, and its Russian surzhyk equivalent is generated with matching morphological features (case, number, gender). This pipeline produced 20,911 high-quality surzhyk examples from approximately 40% of the source corpus.

For all four varieties, dialect pairs were also generated using Mistral-Small-24B-Instruct-2501 running locally with 4-bit NF4 quantization. The DialectCorpusGenerator class orchestrates the pipeline. For each sentence, the DialectPromptBuilder component retrieves the $k = 50$ most relevant dialect dictionary entries via TF-IDF cosine similarity and incorporates them into a dialect-specific system prompt. Sentences were processed in batches of five, with a four-stage quality control pipeline: JSON structure validation, sentence-level format validation, MinHash-based deduplication, and JSONL checkpoint logging for resumable generation. This pipeline produced 108,812 dialect pairs across all four varieties.

The assembled corpus totals 139,576 sentence pairs: 9,853 manual Hutsul annotations, 108,812 LLM-generated pairs, and 20,911 rule-based surzhyk examples. After stratified splitting by dialect, the final distribution is 125,615 training examples, 13,756 validation examples, and 200 test examples (50 hand-annotated per variety, with no overlap with training or validation data).

4. Model Architecture and Training

Dialect normalization is formulated as a conditional sequence generation task. Given a non-standard input sequence $x = (x_1, \dots, x_m)$, the model learns to generate the corresponding standard Ukrainian output $y = (y_1, \dots, y_n)$ by maximizing the conditional log-likelihood over the training corpus:

$$\theta^* = \arg \max_{\theta} \sum_{k=1}^N \log p_{\theta}(y^{(k)} | x^{(k)}). \quad (1)$$

For encoder-decoder models the output is factorized autoregressively conditioned on the full input encoding:

$$p_{\theta}(y | x) = \prod_{t=1}^n p_{\theta}(y_t | x, y_{<t}). \quad (2)$$

The training objective is the cross-entropy loss with label smoothing (smoothing factor $\alpha = 0.1$ for umt5-base):

$$\mathcal{L}(\theta) = - \sum_{t=1}^n \left[(1 - \alpha) \log p_{\theta}(y_t | x, y_{<t}) + \frac{\alpha}{|V|} \sum_{v \in V} \log p_{\theta}(v | x, y_{<t}) \right]. \quad (3)$$

Dialect conditioning is achieved by prepending a dialect identifier prefix to the source sequence: “*Translate from Hutsul: {input}.*” Both models share a single set of parameters across all four varieties, enabling cross-dialect transfer learning.

Both model families rely on the scaled dot-product attention mechanism. Given query matrix Q , key matrix K , and value matrix V :

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^{\top}}{\sqrt{d_k}}\right) V, \quad (4)$$

where d_k is the key dimensionality. Multi-head attention applies h parallel attention heads and projects the concatenated result. The encoder-decoder model uses bidirectional self-attention in the encoder and cross-attention in the decoder; the

decoder-only model uses causal (autoregressive) self-attention throughout.

The first model is based on Google’s mT5-base, a massively multilingual text-to-text transformer with approximately 580M parameters and a SentencePiece vocabulary of 250,112 subword tokens trained on mC4 (101 languages, including Ukrainian). The model was trained with *full fine-tuning* - all parameters updated - for up to 40 epochs with early stopping (patience of 5 epochs on validation BLEU). Training configuration: learning rate 5×10^{-5} with cosine schedule, AdamW optimizer with 8-bit BitsAndBytes quantization of optimizer states, gradient accumulation over 4 micro-batches of size 4 (effective batch size 16), maximum sequence length 256 tokens, BF16 mixed precision. Label smoothing $\alpha = 0.1$ was applied throughout training. The best checkpoint was reached at training step 314,040 (validation BLEU = 71.7).

The second model is based on MamayLM (INSAIT-Institute/MamayLM-Gemma-3-4B-IT-v1.0), a Ukrainian-optimized instruction-tuned variant of Google’s Gemma-3-4B with approximately 4 billion parameters. As a decoder-only autoregressive model, it generates the normalized output token-by-token, framed as an instruction-following chat exchange: the system prompt states the task, and the user turn contains the dialectal input.

To enable fine-tuning within GPU memory constraints, QLoRA (Quantized Low-Rank Adaptation) was applied. The base model weights were quantized to 4-bit NF4 format with double quantization of the quantization constants. Low-rank adapter matrices were added to all attention and feed-forward projection layers. The LoRA update rule is:

$$W' = W + \Delta W = W + BA, \quad B \in \mathbb{R}^{d \times r}, \quad A \in \mathbb{R}^{r \times k}, \quad r \ll \min(d, k), \quad (5)$$

with rank $r = 16$ and scaling factor $\alpha_{LoRA} = 32$. This yields approximately 13 million trainable parameters - 0.4% of the total 4B model.

Training was performed for 3 epochs using the SFTTrainer framework with Unsloth acceleration (sequence packing, asynchronous checkpoint saving).

Configuration: learning rate 2×10^{-4} with cosine schedule and 3% linear warmup, AdamW optimizer, gradient accumulation over 2 steps (effective batch size 2), maximum sequence length 512 tokens, BF16 mixed precision. The cross-entropy loss was applied only to the assistant response tokens (the normalized Ukrainian output), masking the loss on instruction-turn tokens so that the model learns the output distribution without overfitting to the prompt format.

5. Evaluation and Results

Three complementary metrics were used. BLEU (Bilingual Evaluation Understudy) measures n-gram precision between model output and reference, penalizing short outputs via the brevity penalty. chrF++ extends BLEU by incorporating character-level n-grams alongside word n-grams, with $\beta = 2$ weighting favoring recall. chrF++ is used as the primary metric because it is more robust to morphological variation - a common pattern in Ukrainian normalization - than word-level BLEU. TER (Translation Edit Rate) measures the minimum number of edit operations needed to convert model output to the reference, normalized by reference length; lower values are better. All metrics are computed using the SacreBLEU library.

The test set consists of 200 manually annotated sentence pairs (50 per variety), held out from both training and validation. All test examples are human-annotated (not LLM-generated), ensuring a reliable evaluation signal independent of the synthetic data quality. Three commercial large language model baselines were evaluated in zero-shot mode via the OpenRouter API: GPT-4o, Gemini 2.0 Flash, and Mistral Large. Each baseline received the same dialect prefix prompt as the fine-tuned models, with no in-context examples. Additionally, MamayLM in its pre-fine-tuning state was evaluated to isolate the contribution of fine-tuning from the model’s pretraining knowledge.

On surzhyk, both fine-tuned models substantially outperform all zero-shot baselines. umt5-base achieves chrF++ = 93.58 versus Gemini’s 86.67 (the strongest baseline), a margin of 6.9 points. MamayLM before fine-tuning scores 65.58; after

QLoRA training it reaches 91.22 - a gain of 25.6 points. The strong performance is explained by the large training corpus ($\approx 50,500$ pairs) and the lexical proximity of surzhyk to standard Ukrainian: normalization requires mainly lexical and morphological substitution, for which the combined rule-based and LLM-synthetic data is representative.

On the Hutsul dialect, umt5-base achieves $\text{chrF++} = 86.90$, outperforming Gemini 2.0 Flash (65.34) by 21.6 points. The improvement from pre-training to post-training for MamayLM is $16.73 \rightarrow 74.02$, an absolute gain of 57.3 points - the largest observed for any model-variety combination. This dramatic improvement demonstrates the decisive role of the 9,853 manually annotated pairs: they provide authentic phonological patterns that the pre-trained model had not encountered during general-purpose training, and that LLM-generated data alone cannot replicate reliably.

On the Boikivian dialect, commercial large language models outperform the fine-tuned models. Gemini 2.0 Flash achieves $\text{chrF++} = 74.57$, compared to MamayLM fine-tuned (66.82) and umt5-base (54.78). The training data for Boikivian is entirely LLM-generated (29,754 pairs), and the generation model may not have replicated authentic Boikivian phonology with sufficient fidelity. Commercial LLMs with far larger pretraining corpora appear to have encountered more genuine Boikivian-like text than the synthetic training data captures.

The Transcarpathian dialect is the most challenging variety for all systems. Fine-tuned models score far below zero-shot baselines: umt5-base 27.45, MamayLM fine-tuned 20.93, versus Gemini 2.0 Flash 68.16. The Transcarpathian dialect has the greatest linguistic distance from the literary norm, with systematic vowel shifts, contact borrowings from three neighboring languages, and phonological features largely absent from other Ukrainian varieties. The training data is entirely synthetic (29,605 pairs), and the distribution mismatch is most severe here. The fact that pre-fine-tuning MamayLM (27.90) marginally outperforms the fine-tuned version (20.93) suggests that low-quality synthetic training data can slightly degrade performance on

genuinely out-of-distribution patterns.

Specialized fine-tuning on quality annotated data decisively outperforms zero-shot large language model prompting for varieties where such data is available (surzhyk, Hutsul). For varieties where training data is purely synthetic (Boikivian, Transcarpathian), commercial large language models with broader pretraining retain a clear advantage. The primary bottleneck for dialect normalization is not model capacity or architecture, but the availability of manually annotated parallel data.

6. The “Leksykon” Web Application

The “Leksykon” system is a web application providing interactive access to the normalization system for both typed and spoken dialectal Ukrainian input. It was implemented using NiceGUI 1.4+, a Python-native UI framework built over FastAPI/ASGI, enabling the normalization and ASR models to be co-located in the same Python process. The application is launched via make demo and is accessible at <http://0.0.0.0:7870>. A CPU fallback mode (float32 precision) is available when no GPU is present.

Normalization module. The DialectTranslator class loads the fine-tuned umt5-base-multidialect model at startup in float16 precision on GPU. Normalization uses beam search with num_beams=5, max_length=128, repetition_penalty=1.2, and no_repeat_ngram_size=3 to discourage repetitive outputs. The user can interactively adjust beam width (3-10) and repetition penalty (1.0-2.0) via UI sliders without reloading the model, enabling quality-speed trade-off exploration.

Speech recognition module. The ASR module uses the SpeechBrain m-ctc-t-large model (MCTCTForCTC architecture, multilingual, pretrained on seven languages including Ukrainian). Audio is recorded in the browser via the MediaRecorder API and sent to the FastAPI endpoint POST /api/transcribe. The PyAV library decodes the audio and resamples it to 16 kHz mono float32 format. Supported audio formats: WebM, OGG, WAV. The recognized transcript is populated into the text input field, where it can be reviewed and submitted for normalization.

User interface. The application uses a three-column desktop layout with a

vertical stack on mobile devices (viewport < 768 px). The interface includes a dialect selector (four buttons: Hutsul, Boikivian, Transcarpathian, Surzhyk), a text input field with a microphone button for voice input, a normalize button, an output field, adjustment sliders, and four preset example sentences per dialect for guided testing. The complete frontend is implemented in approximately 250 lines of Python using NiceGUI components.

7. Conclusions and Future Work

The work achieved the following main results:

1. An analysis of existing approaches to dialect normalization and grammatical error correction identified the absence of publicly available parallel corpora for the Boikivian and Transcarpathian dialects. No prior system addresses surzhyk and three Ukrainian dialects within a unified normalization framework.
2. A manual Hutsul corpus of 9,853 sentence pairs was incorporated, and the Spivavtor corpus was processed as a source of high-quality standard Ukrainian target sentences.
3. A rule-based surzhyk generation pipeline (SurzhykErrorifier) was implemented using pymorphy2 morphological analysis and a surzhyk substitution lexicon, producing 20,911 training examples.
4. An LLM-based dialect generation pipeline (DialectCorpusGenerator with Mistral-Small-24B-Instruct-2501, TF-IDF dictionary retrieval, and four-stage quality control) produced 108,812 synthetic training pairs across all four varieties.
5. The umt5-base encoder-decoder model was trained to convergence over 40 epochs (with early stopping), achieving chrF++ = 93.58 on surzhyk and 86.90 on the Hutsul dialect - both substantially above all zero-shot large language model baselines.
6. The MamayLM (Gemma-3-4B) decoder-only model was fine-tuned with QLoRA ($r = 16$, $\alpha_{LoRA} = 32$), updating only 0.4% of the model's

parameters. It achieved chrF++ = 91.22 on surzhyk and 66.82 on Boikivian, outperforming umt5-base on the latter dialect.

7. The “Leksykon” web application integrating umt5-base normalization with SpeechBrain ASR was implemented and deployed.

The central finding of this work is that specialized fine-tuning on quality annotated data significantly outperforms zero-shot large language model prompting for dialect normalization - but only when that annotated data is available. For Boikivian and Transcarpathian, where training data is entirely synthetic, commercial large language models with broader pretraining exposure retain a clear advantage. This result reframes the core challenge of Ukrainian dialect NLP: the bottleneck is not model architecture or compute, but the availability of manually annotated parallel data.

Future work should prioritize: (1) collection of 3,000-5,000 manual parallel pairs for Boikivian and Transcarpathian dialects, which is expected to substantially close the performance gap with commercial baselines; (2) fine-tuning of the ASR module on dialectal speech recordings to improve recognition accuracy for non-standard input; (3) extension of the normalization system to additional Ukrainian dialects (Slobozhan, Podilian, Volhynian); and (4) development of a REST API for integration with downstream natural language processing pipelines.

ЗМІСТ

АНОТАЦІЯ.....	4
ABSTRACT	5
ЗМІСТ.....	17
ПЕРЕЛІК УМОВНИХ ПОЗНАЧЕНЬ	20
ВСТУП.....	22
1. ЗАГАЛЬНІ ПОЛОЖЕННЯ	25
1.1. Аналіз предметної галузі та літературних джерел	25
1.1.1. Нормалізація як задача між нейронним машинним пере- кладом і виправленням граматичних помилок.....	25
1.1.2. Діалекти та суржик як об'єкти автоматичної нормалізації	26
1.1.3. Наявні ресурси та підходи до синтетичного розширення да- них	27
1.1.4. Сучасні україномовні моделі та найближчі суміжні роботи ..	28
1.2. Аналіз вхідних даних.....	29
1.2.1. Лінгвістична характеристика мовних різновидів	30
1.2.2. Ресурси для гуцульського діалекту	31
1.2.3. Корпус «Співавтор» як джерело літературної норми	32
1.2.4. Словникова база мовних різновидів	33
1.2.5. Синтетична генерація даних	34
1.2.6. Формування та структура фінального корпусу	35
1.3. Висновки до розділу	36
2. ІНФОРМАЦІЙНЕ ТА МАТЕМАТИЧНЕ ЗАБЕЗПЕЧЕННЯ	38
2.1. Аналіз підходів до нормалізації.....	38
2.1.1. Нормалізація як задача перетворення тексту	38
2.1.2. Правильні підходи та їх обмеження.....	39

2.1.3. Нейромережеві підходи: кодер-декодерна модель і декодерна BMM.....	39
2.2. Постановка задачі та математична модель.....	40
2.2.1. Формальна постановка нормалізації.....	40
2.2.2. Формат даних для двох архітектур	41
2.2.3. Цільова функція: крос-ентропія і згладжування міток	41
2.3. Математичні основи трансформера.....	42
2.3.1. Механізм уваги	42
2.3.2. Позиційне кодування, нормалізація шарів і прямозв'язний блок.....	42
2.3.3. Кодер-декодерна архітектура.....	43
2.3.4. Авторегресійна декодерна модель	45
2.4. Вибір та обґрунтування моделей.....	45
2.4.1. Кодер-декодерна модель umt5-base.....	45
2.4.2. Декодерна BMM MamyLM.....	46
2.4.3. Параметро-ефективне донавчання: LoRA та QLoRA	46
2.5. Гіперпараметри та конфігурація навчання.....	47
2.5.1. Доновчання umt5-base.....	47
2.5.2. Доновчання MamyLM (QLoRA).....	49
2.6. Метрики оцінювання	50
2.6.1. BLEU.....	50
2.6.2. chrF++	50
2.6.3. TER	51
2.6.4. Взаємодоповнюваність метрик.....	51
2.7. Стратегії декодування	52
2.8. Балансування даних	52
2.8.1. Дисбаланс діалектів та принципи семплювання	52
2.8.2. Формат інструкцій для мультидіалектності	53
2.9. Висновки до розділу	53

3. ПРОГРАМНЕ ТА ТЕХНІЧНЕ ЗАБЕЗПЕЧЕННЯ	54
3.1. Технологічний стек.....	54
3.2. Конвеєр генерації корпусу.....	55
3.2.1. Правильна генерація суржику	55
3.2.2. LLM-генерація для діалектів	57
3.2.3. Підготовка корпусу до навчання.....	58
3.3. Навчання моделей.....	59
3.3.1. Донавчання umt5-base	59
3.3.2. Донавчання MamayLM.....	60
3.4. Оцінювання та аналіз результатів.....	63
3.4.1. Методологія оцінювання.....	63
3.4.2. Результати кодер-декодерної моделі umt5-base	63
3.4.3. Результати декодерної ВММ MamayLM	64
3.4.4. Порівняння з нуль-шотовими базовими лініями	65
3.4.5. Аналіз результатів.....	65
3.5. Вебзастосунок «Лексикон»	67
3.5.1. Архітектура застосунку.....	67
3.5.2. Модуль нормалізації	68
3.5.3. Модуль автоматичного розпізнавання мовлення	69
3.5.4. Інтерфейс користувача	69
3.6. Висновки до розділу	70
ЗАГАЛЬНІ ВИСНОВКИ	72
СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ.....	74
Додаток А.....	76

ПЕРЕЛІК УМОВНИХ ПОЗНАЧЕНЬ

APM - автоматичне розпізнавання мовлення (Automatic Speech Recognition) - технологія перетворення мовленнєвого сигналу на текст.

BMM - велика мовна модель (Large Language Model) - нейронна модель на основі трансформера, навчена на великих масивах тексту та здатна до генерації і обробки природної мови.

ГП - графічний процесор - спеціалізований процесор для паралельних обчислень, що використовується при навчанні нейронних мереж.

НМП - нейронний машинний переклад (Neural Machine Translation) - підхід до автоматичного перекладу тексту з використанням нейронних мереж.

ОП - оперативна пам'ять - основна пам'ять комп'ютера для зберігання даних під час виконання програм.

ОПМ - обробка природної мови (Natural Language Processing) - галузь штучного інтелекту, що вивчає методи автоматичного аналізу і генерації текстів.

ЦП - центральний процесор - основний обчислювальний пристрій комп'ютера.

API (Application Programming Interface) - програмний інтерфейс застосунків, набір визначень і протоколів для взаємодії між програмними компонентами.

BLEU (Bilingual Evaluation Understudy) - автоматична метрика оцінювання якості машинного перекладу на основі збігу n-грамів.

chrF++ (character n-gram F-score) - метрика оцінювання якості перекладу на основі символічних і словесних n-грамів.

LoRA (Low-Rank Adaptation) - метод параметро-ефективного донавчання нейронних мереж шляхом введення низькорангових матриць адаптації.

PEFT (Parameter-Efficient Fine-Tuning) - підхід до донавчання великих нейронних мереж, при якому оновлюється лише невелика частина параметрів.

QLoRA (Quantized LoRA) - варіант методу LoRA із квантизацією ваг ба-

зової моделі до 4 біт для зменшення вимог до пам'яті.

TER (Translation Edit Rate) - метрика оцінювання якості перекладу, що вимірює кількість редагувань, необхідних для приведення результату до еталонного.

ВСТУП

Актуальність теми. Українська мова існує в умовах значної варіативності: поряд із літературною нормою в різних регіонах побутують гуцульський, бойківський і закарпатський діалекти, а в містах широко вживається суржик - змішане українсько-російське мовлення. Масова внутрішня міграція після початку повномасштабного вторгнення 2022 року поглибила мовні контакти: люди з різними мовними звичками опиняються в одному середовищі, де діалектні та суржикові форми можуть ускладнювати ділову комунікацію, навчання і роботу з документами. Попри очевидну потребу, спеціалізованих автоматичних інструментів нормалізації регіональних діалектів і суржику в Україні майже немає: наявні системи орієнтовані переважно на орфографічні та граматичні помилки і не розраховані на систематичне перетворення діалектних форм на літературну норму.

Потреба в нормалізації виходить за межі суто лінгвістичного інтересу. Перетворення діалектного або суржикового тексту на літературний стандарт корисне в освіті, редакційній підготовці документів, а також як проміжний крок у системах обробки природної мови, навчених переважно на нормативних текстах. Сучасні великі мовні моделі добре опрацьовують літературну українську, проте для регіональних діалектів якість різко падає: таких текстів обмаль у відкритих джерелах, тому моделі не мають достатнього сигналу для навчання. Без попередньої нормалізації діалектний вхід погіршує роботу пошукових систем, голосових інтерфейсів і засобів автоматичного аналізу тексту.

Ключовою перешкодою залишається брак паралельних даних. Для гуцульського діалекту існують поодинокі ручні корпуси, а для бойківського і закарпатського публічних паралельних корпусів немає взагалі. Суржик додатково ускладнює задачу відсутністю усталеної норми і регіональної прив'язки. Це зумовлює потребу у власних паралельних корпусах і спеціалізованих моделях, які враховують регулярні фонетичні, лексичні та морфологічні особливості кожного різновиду, а не лише орфографічні помилки. Саме на розв'язання цієї

проблеми спрямована робота.

Мета роботи - розробка та дослідження системи автоматичної нормалізації суржику і трьох регіональних діалектів (гуцульського, бойківського, закарпатського) до літературного стандарту сучасної української мови на основі нейронних моделей і підходів до роботи з малоресурсними даними.

Завдання дослідження:

- проаналізувати сучасні підходи до автоматичної нормалізації діалектних і гібридних мовних форм;
- обґрунтувати стратегію формування паралельного корпусу для чотирьох мовних різновидів з обмеженим обсягом ручної розмітки;
- зібрати і підготувати паралельний корпус для гуцульського, бойківського, закарпатського діалектів і суржику;
- навчити та порівняти дві моделі нормалізації - кодер-декодерну (umt5-base) і декодерну велику мовну модель (MamayLM) - на спільному багатодіалектному корпусі;
- оцінити якість навчених моделей у порівнянні з нуль-шотовими великими мовними моделями за метриками BLEU, chrF++ і TER;
- реалізувати прикладний вебзастосунок «Лексикон» із підтримкою голосового введення.

Об'єктом дослідження є процес автоматичної нормалізації нелітературних форм сучасної української мови - суржику та регіональних діалектів.

Предметом дослідження є методи формування паралельних корпусів для малоресурсних мовних варіантів, а також методи навчання нейронних моделей нейронного машинного перекладу і великих мовних моделей для задачі нормалізації.

Методи дослідження: аналіз і формування корпусів текстових даних; синтетична генерація паралельних прикладів за допомогою лінгвістичних правил, словникових ресурсів і великої мовної моделі; навчання кодер-декодерних трансформерних моделей; параметро-ефективне донавчання великих мовних

моделей (QLoRA); автоматичне оцінювання якості за метриками BLEU, chrF++ і TER.

Наукова новизна роботи полягає в тому, що вперше запропоновано і досліджено єдину систему нормалізації одночасно для суржику і трьох регіональних діалектів. Для бойківського та закарпатського діалектів сформовано перші публічні синтетичні паралельні корпуси. Проведено порівняльне оцінювання кодер-декодерної та декодерної архітектур на задачі мультидіалектної нормалізації з залученням нуль-шотових великих мовних моделей як базових ліній.

Практична значущість роботи полягає у розробці вебзастосунку «Лексикон», що нормалізує тексти гуцульського, бойківського і закарпатського діалектів та суржику до літературної української мови з підтримкою голосового введення. Систему можна використовувати як самостійний інструмент або інтегрувати до ширших конвеєрів автоматичної обробки україномовних текстів.

Особистий внесок. Усі результати роботи отримано автором особисто: зібрано і впорядковано паралельний корпус для чотирьох мовних різновидів, реалізовано правилний і генеративний конвеєри побудови синтетичних даних, виконано донавчання та оцінювання кодер-декодерної моделі umt5-base і декодерної моделі MatayLM, розроблено вебзастосунок «Лексикон» із текстовим і голосовим введенням, а також підготовлено пояснювальну записку.

1 ЗАГАЛЬНІ ПОЛОЖЕННЯ

1.1 Аналіз предметної галузі та літературних джерел

1.1.1 Нормалізація як задача між нейронним машинним перекладом і виправленням граматичних помилок

Автоматична нормалізація діалектних форм і суржику до літературної норми є задачею, що поєднує властивості двох напрямів обробки природної мови (ОПМ): нейронного машинного перекладу (НМП) та автоматичного виправлення граматичних помилок. Від НМП вона відрізняється тим, що перетворення відбувається між різновидами однієї мови: фонологічна, лексична та морфологічна системи діалекту частково збігаються з літературною нормою, а вхідний текст зазвичай залишається зрозумілим для носія стандарту. Від задачі виправлення граматичних помилок нормалізація відрізняється тим, що допустимі відхилення є системними - правильними в межах певного діалекту - а не випадковими помилками: регулярні фонетичні та лексичні трансформації не можна просто «виправити», не враховуючи діалектний контекст.

На практиці задача нормалізації може розв'язуватися трьома основними підходами, що еволюціонували від правилкових систем до нейронних. Правильові та словникові системи покладаються на явно задані відповідники між діалектними формами і літературними: словники заміни, регулярні перетворення морфем, фонетичні правила. Такі системи добре інтерпретовані, але погано масштабуються на нові домени і не охоплюють контекстно-залежних трансформацій. Моделі типу «послідовність-у-послідовність» на основі трансформера дозволяють навчити кодувально-декодуєчу модель на паралельних даних «діалект → норма» і природно трактують задачу як переклад усередині мови. Великі мовні моделі (ВММ) з інструкційним донавчанням поєднують генеративну потужність авторегресійних моделей і гнучкість формату запиту, що дозволяє одній моделі нормалізувати кілька різновидів за умови явної вказівки типу задачі.

Роботи з НМП для діалектів підтверджують, що багатозадачне навчан-

ня, яке об'єднує кілька мовних різновидів в одну модель, дає кращі результати за умови обмежених даних: для арабських діалектів показано, що спільна модель стабілізує навчання і узагальнює спільні лінгвістичні закономірності [1.]. Для малоресурсних сценаріїв принципово важливо, що навіть невеликий обсяг якісних паралельних даних суттєво покращує якість відносно нуль-шотового застосування загальної моделі. Байтові підходи, як-от ВуТ5, зменшують залежність від словникових сегментацій і краще працюють із нестандартизованими текстами, що містять фонетичні відхилення [15.]. Для багатомовних сценаріїв важливою є також стратегія збалансованого вибору мов під час попереднього навчання, що впливає на якість для малоресурсних мов [3.].

1.1.2 Діалекти та суржик як об'єкти автоматичної нормалізації

Чотири мовні різновиди, що досліджуються в цій роботі, суттєво відрізняються за природою і ступенем систематизованості відхилень від літературної норми.

Гуцульський діалект поширений у гірських районах Карпат - Івано-Франківській, Закарпатській та Чернівецькій областях. Він є добре задокументованим: для нього наявні лінгвістичні описи, словники та паралельні корпуси. Гуцульський вирізняється специфічними голосними змінами (наголошені «а» та «я» переходять у «є»: «як» → «єк», «шапка» → «шепка»), особливою поведінкою зворотної частки (використовується «си» замість «ся» і може стояти перед дієсловом: «він сміється» → «він си сміє»), а також характерними відмінами дієслів третьої особи однини («знає» → «знаєт»). Наявність ручних паралельних даних і словника зробила гуцульський природною відправною точкою для побудови системи.

Бойківський діалект є одним із найархаїчніших в Україні - він зберігає риси давньоукраїнської мови і праслов'янські фонологічні елементи. Характерні ознаки: розрізнення переднього [и] і заднього [ы], що зникло в стандартній мові («бик» → «бык», «тихо» → «тыхо»); перехід [л] у [љ] у кінці складу («галка» → «гавка», «кіл» → «ків»); особові енклітики в умовному способі (-бим, -

бись, -бисьмо); архаїчні числівникові форми («дев'ятнадцять» → «двайцять без єнного»). Бойківський є найбільш лексично задокументованим серед чотирьох різновидів.

Закарпатський діалект вирізняється тривалим контактом із сусідніми мовами - словацькою, угорською та румунською. Збережено архаїчний звук [ы] як безпосередній продовжувач праслов'янського («мити» → «мыти», «дрова» → «дрыва»); характерні вокальні зсуви у закритих складах ([o] → [y]: «кінь» → «кунь»); запозичені частки угорського і румунського походження. Закарпатський - найскладніший для автоматичної нормалізації: велика лінгвістична відстань від норми і мала кількість ручних паралельних даних утруднюють навчання.

Суржик є принципово іншим явищем - це не регіональний діалект, а соціолінгвістичний феномен змішаного мовлення, поширений у центральних, східних і південних регіонах України. Носії суржику змішують лексичні, морфологічні та фонетичні елементи з обох мов, часто без чіткої регіональної прив'язки [6.]. Автоматичне розпізнавання суржику розглянуто в [13.]. Типові ознаки: лексичні заміни (але → но, дуже → очень, зупинка → остановка), зміна закінчень інфінітива (-ти → -ть: «робити» → «робить»), сполучникові кальки (якщо → если). Оскільки суржик не має єдиної фонологічної системи, його нормалізація потребує покриття широкого спектра лексичних і морфологічних замін.

1.1.3 Наявні ресурси та підходи до синтетичного розширення даних

Для задачі виправлення граматичних помилок в українській мові сформувалися перші систематичні ресурси. UA-GEC [14.] - перший публічний корпус для граматичної корекції та нормалізації флюентності в українській мові, що охоплює анотацію за типами помилок і дає базу для порівняльного аналізу методів. Багатомовний корпус OmniGEC [8.] розширив задачу на міжмовний рівень, запропонувавши срібну розмітку для кількох мов, включаючи українську. Інструкційно-донавчена модель «Співавтор» [12.] охоплює виправлення поми-

лок, перефразування, спрощення та покращення стилю в українських текстах і є одним із ключових джерел навчальних прикладів у цій роботі.

Діалектна нормалізація перетинається з цими задачами в частині вибору формату даних і методів навчання, але суттєво від них відрізняється: замість «помилки» маємо систематичні трансформації, правильні в межах діалекту. Критичним обмеженням є малоресурсність: для бойківського і закарпатського діалектів ручних паралельних даних майже не існує, а для суржику відсутній усталений стандарт розмітки.

Стандартним рішенням у малоресурсних сценаріях є синтетичне розширення корпусу. Дослідження синтетично згенерованих даних для українського виправлення граматичних помилок [2.] підтверджує, що якість синтетики суттєво залежить від правил навмисного введення відхилень і що моделі, навчені на синтетиці, можуть конкурувати з моделями на ручних даних за достатньої різноманітності прикладів. У цій роботі застосована схема «норма → нестандарт»: за відправну точку береться правильне літературне речення, після чого генерується його діалектний або суржиковий варіант. Перевага такого напрямку полягає в тому, що еталон гарантовано коректний від початку. Для суржику застосовано два незалежні підходи: правилловий через морфологічний аналізатор і генеративний за допомогою ВММ; для регіональних діалектів - лише генеративний із лінгвістичними правилами.

1.1.4 Сучасні україномовні моделі та найближчі суміжні роботи

Напрямок адаптації сучасних нейронних архітектур до української мови активно розвивається. Робота «Bytes to Borsch» [7.] дослідила донавчання моделей Gemma і Mistral для покращення україномовної репрезентації і показала, що цільове донавчання навіть порівняно невеликого обсягу помітно підвищує якість для задач розуміння і генерації. Відкрита україномовна ВММ LapaLLM [10.] побудована на базі Gemma-3-12B і оптимізована під українську через адаптацію токенизатора. MamaLM [11.] - менша за параметрами (4 млрд), проте за результатами на стандартних тестах конкурентна модель, яка орієнто-

вана на практичне застосування і доступна через відкриту ліцензію. Для задачі підтримки україномовних текстів загалом важливими є також розвиток ресурсної бази мови [5.] та поєднання математичних моделей і машинного навчання для виявлення і виправлення помилок [4.]. Актуальним є і напрям оцінювання якості моделей від ручних ознак до великих мовних моделей [16.].

Найближчою до нашої постановки є робота «Vuyko Mistral: Adapting LLMs for Low-Resource Dialectal Translation» [9.]: у ній запропоновано методологію адаптації ВММ до малоресурсних діалектних сценаріїв, зокрема принципи формування паралельних корпусів і способи донавчання. Разом із тим, ця робота зосереджена на задачі перекладу з діалекту на інші мови, а не на нормалізації до єдиного літературного стандарту усередині мови. Крім того, набір досліджуваних різновидів і суржик як окреме явище в ній не охоплені. Запропонована нами система відрізняється: вона спрямована на нормалізацію чотирьох різновидів - гуцульського, бойківського, закарпатського діалектів і суржику - до літературного стандарту і побудована на власному паралельному корпусі, сформованому комбінацією ручної розмітки і синтетичної генерації.

Аналіз літератури засвідчує три ключових висновки: нормалізація діалектів і суржику методично близька до НМП і виправлення граматичних помилок, але потребує спеціалізованих ресурсів; за браку ручних даних синтетичне розширення через правила і ВММ є практичним рішенням; для українських діалектів і суржику публічних паралельних корпусів і спеціалізованих моделей нормалізації до цього часу не існувало.

1.2 Аналіз вхідних даних

Для задачі автоматичної нормалізації якість і репрезентативність навчальних даних є визначальними. На відміну від класичного НМП між різними мовами, тут маємо справу з мовними різновидами однієї мови, де межа між «помилкою», «локальною нормою» та «варіантом написання» часто розмита. Тому до корпусу ставляться додаткові вимоги: узгоджені еталони, прозорий поділ на навчальну, валідаційну та тестову вибірки, контроль джерел (ручні чи синтетичні

пари) і баланс між чотирма мовними різновидами.

У роботі застосовано комбінований підхід: як опорну базу взято ручні паралельні дані для гуцульського діалекту; для решти діалектів і суржику корпус сформовано синтетично - за допомогою лінгвістичних правил і великої мовної моделі; як джерело різноманітних літературних цільових речень для суржику використано корпус «Співавтор» [12.].

1.2.1 Лінгвістична характеристика мовних різновидів

Для побудови ефективного синтетичного генератора і розуміння природи труднощів задачі нормалізації необхідно систематизувати лінгвістичні особливості кожного мовного різновиду.

Гуцульський діалект. На рівні голосних: наголошені «а» і «я» регулярно переходять у «є» («як» → «єк», «шапка» → «шєпка», «яблуко» → «єблуко»); «і» в окремих позиціях переходить у «и» («Іван» → «Иван», «іду» → «иду»); «ю» переходить у «у» («вівцю» → «вівцу»). На рівні приголосних: зворотна частка змінюється з «ся» на «си» і може стояти перед дієсловом («він сміється» → «він си сміє»); сибілянти пом'якшуються («чого» → «чьо»); «ч» та «ц» можуть замінювати одна одну («чи» → «ци»); групи «дн», «тн», «лн» спрощуються до «нн» або «н» («передній» → «перенний»). На морфологічному рівні: дієслова третьої особи однини отримують закінчення -єт («знає» → «знаєт», «має» → «маєт»); у минулому часі можливі аналітичні форми з енклітиками («ходив» → «ходив'єм»); прийменник «від» переходить у «вид»; заперечна частка «не» переходить у «ни» («не знаю» → «ни знаю»).

Бойківський діалект. Є одним із найархаїчніших в Україні і зберігає риси давньоукраїнської мови. Головна фонологічна риса - розрізнення переднього [и] і заднього [ы], яке зникло в стандартній мові: «бик» → «бык», «тихо» → «тыхо», «дівки» → «дівкы». Інша характерна риса - перехід [л] у [љ] у кінці складу: «галка» → «гавка», «кіл» → «ків». На морфологічному рівні: спрощення груп приголосних («бідна» → «бінна», «весілля» → «весіля»); особові енклітики в умовному способі («ходив би» → «ходивбим»); архаїчна форма третьої

особи множини «суть»; унікальні числівникові конструкції («дев'ятнадцять» → «двайцять без єнного»); м'яке вимовляння сибілянтів ([ж], [ш], [ч]). Словник бойківського є найобширнішим серед чотирьох різновидів і містить 14 910 записів.

Закарпатський діалект. Визначається поєднанням архаїчних праслов'янських рис і запозичень із сусідніх мов. Архаїчний задній голосний [ы] збережено як безпосередній продовжувач праслов'янського звуку: «мити» → «мыти», «синок» → «сынок», «дрова» → «дрыва». Характерні вокальні зсуви у закритих складах: [о] переходить у [у] або [ү] («кінь» → «кунь», «стіл» → «сціл»). З угорської запозичено порівняльну частку «тај» і розділовий сполучник «вад'»; з румунської - «хот'» для поступки. Займенникова система відрізняється від стандартної: демонстративні займенники «тот»/«тота» замість «той»/«та». Велика лінгвістична відстань між закарпатськими формами і літературною нормою обумовлює найнижчу якість нормалізації серед чотирьох різновидів.

Суржик. На відміну від діалектів, суржик не є регіональним явищем зі сталою фонологічною системою: кожен носій демонструє власний ступінь і спосіб змішування мов [6.]. Водночас існують повторювані риси: лексичні заміни (але → но, дуже → очень, зупинка → остановка, майже → почті); зміна закінчення інфінітива («робити» → «робить», «ходити» → «ходить»); сполучникові кальки («якщо» → «єслі», «тому що» → «потому шо»); дискурсні маркери («ось» → «вот», «звичайно» → «конєшно»). Стратегія синтетичної генерації суржику полягала у застосуванні 1-3 типових заміन на речення для отримання природного, а не карикатурного змішування.

1.2.2 Ресурси для гуцульського діалекту

Гуцульський діалект обрано відправною точкою, бо для нього доступні найбільш повні ресурси: вручну анотований паралельний корпус, синтетично згенерований корпус і діалектний словник (рис. 1.1). Ручний корпус містить 9 853 пари «гуцульська форма → літературний відповідник» і є основою для

формування тестової вибірки - саме на цих даних метрики відображають реальну якість системи, а не якість генератора синтетики.

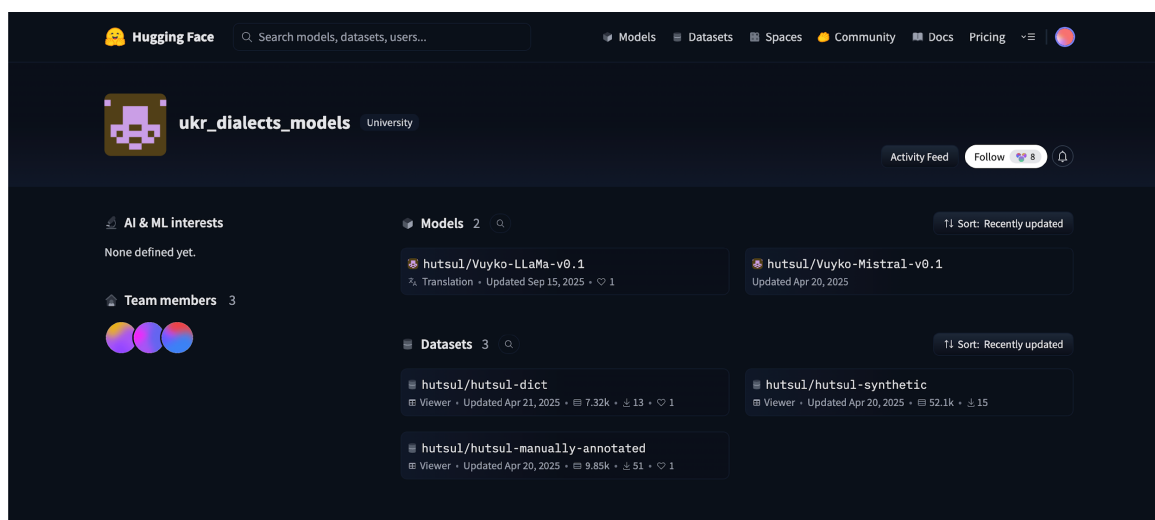


Рис. 1.1: Гуцульські ресурси на платформі Hugging Face: датасети та допоміжні матеріали, що використовувались як вихідні джерела.

Ручний корпус доповнено синтетично згенерованими парами: за допомогою моделі Mistral Small 24B з лінгвістичними правилами гуцульського діалекту отримано 19 841 пару. Таким чином, загальний обсяг гуцульської підвибірки в навчальному наборі склав 29 438 прикладів.

Діалектний словник виконував кілька функцій: слугував джерелом лексичних відповідників під час синтетичної генерації, обмежував простір допустимих діалектних замін і використовувався для валідації згенерованих пар - якщо у вхідному тексті були діалектні слова, перевірялась їх наявність у словнику. Словник містить 7 320 записів у форматі «діалектна форма - стандартна форма - словникова лема».

На рис. 1.2 показано приклад вмісту вручну анотованого корпусу: кожен рядок містить вхідний діалектний текст і відповідний літературний еталон. У задачі нормалізації один діалектний вираз може мати кілька допустимих літературних варіантів, тому ручна розмітка фіксує один узгоджений еталон.

1.2.3 Корпус «Співавтор» як джерело літературної норми

Для суржику еталонні речення літературної норми взято з корпусу «Співавтор» (рис. 1.3) - набору пар редагування і перефразування українською мовою,

Dataset Viewer Auto-converted to Parquet API Embed Duplicate Data Studio

Split (1)
train · 9.85k rows

Search this dataset

target string · lengths	source string · lengths
Всій пропій був на конех, буйних, золотогрих та грудистих.	Весь пропій був на конях – буйних, золотогрих і грудистих.
До надвірної роботи уни си нам ни чепали.	До надвірної роботи вони не бралися.
Ти пий аді воду.	Ти пий, ади, воду.
Тай пишов з тим усім надвір.	І пішов з тим усім надвір.
Хліви, що наоколо оповивали иззаду хати и обі кліти з боків, запахом овечього гною дихали.	Хліви, що наоколо оповивали ззаду хати й обидві кліті з боків, дихали запахом овечого гною.
З глибокого ізвора долітав до мене вічний шум потоків.	З глибокого звору долітав до мене вічний шум потоків.
Таже най ти доповістуй лиш за Олексія, то зараз уздриш, ек то є.	Таж розповім лиш про Олексія, то зараз побачиш, як то є.

Size of downloaded dataset files: 3.71 MB
Size of the auto-converted Parquet files: 1.96 MB Number of rows: 9,852

Models trained or fine-tuned on hutsul/hutsul-...

hutsul/Vuyko-LLaMa-v0.1
Translation · Updated Sep 14, 2025 · 1

Рис. 1.2: Вміст вручну анотованого паралельного корпусу для гуцульського діалекту: стовпці вхідного тексту та еталонного відповідника.

що охоплює виправлення граматичних помилок, перекладування і спрощення тексту [12.]. У контексті цієї роботи «Співавтор» використовувався не як діалектний ресурс, а як пул стилістично природних літературних речень: з нього відбирались цільові речення, на основі яких генерувався суржиковий варіант.

Практична логіка така: обирається літературне речення y зі «Співавтора», після чого генерується суржиковий варіант x . Таким чином пара (x, y) гарантовано має якісний еталон. Перевага такого підходу полягає ще й у тому, що «Співавтор» містить широкий спектр природних речень із різних доменів - новини, побутові тексти, публіцистика, - що забезпечує доменну різноманітність навчальних прикладів для суржику.

1.2.4 Словникова база мовних різновидів

Для кожного з чотирьох мовних різновидів зібрано словники відповідників між діалектними і стандартними формами. Словники виконували подвійну роль у системі: слугували джерелом прикладів для шаблонів запитів до ВММ та засобом контролю якості згенерованих пар. Основні характеристики словників наведено в таблиці 1.1.

Бойківський словник є найбільшим (14 910 записів), що відображає значну лексичну відстань між бойківськими формами і стандартом. Словник суржику суттєво менший (570 записів), оскільки основні відхилення суржику є

Datasets: [grammarly/spivavtor](#) like 5 Follow Grammarly 200 Dataset card

Split (2)
train · 62.8k rows

Search this dataset

id	src	tgt	task
int64	string · lengths	string · lengths	string · classes
1e6.28k	10%	99*176 34.3%	77*153 29.8%
			44.5%
1	Перефразуйте: Кайс став одержимий нещодавно усвідомленою вразливістю.	Кайс був одержимий своєю нововиявленою вразливістю.	paraphrase
2	Перепишіть речення іншими словами: І вона більше не думатиме про Джонні Сміта та його...	Джонні Сміт і його дивна чарівна посмішка, про яку він ніколи більше не згадав.	paraphrase
3	Виправити граматику: 3 простими, там, поїсти, поспати не важко.	3 простими, там, поїсти, поспати не важко.	grec
4	Виправте всі граматичні помилки: Тоді як ДНК має дві спіралі – інформаційний ряд...	Тоді як ДНК має дві спіралі – інформаційний ряд з'єднаний з комплементарним йому іншим...	grec
5	Зробіть цей текст легше для розуміння: Ви в дуже серйозній біді, містере Штайнфелд.	Містере Штайнфелд, у вас великі проблеми.	simplification
6	Виправте зв'язність в цьому тексті: англіканство вперше прийшло до Бразилії на...	Англіканство вперше прибуло до Бразилії на початку 19 століття, коли португальська...	coherence
7	Покращте зв'язність тексту: пісні у стилі блек-метал часто відрізняються від...	Блек-метал-пісні часто відрізняються від традиційної структури пісень і часто не...	coherence
8	Виправити граматику: Іван побачив сірий берет в гуші на початку Великої Микитської,...	Іван побачив сірий берет у гуші на початку Великої Микитської, але Герцена.	grec

< Previous 1 2 3 ... 628 Next >

Рис. 1.3: Приклад вмісту корпусу «Співавтор»: вхідний і відредагований текст із міткою типу завдання.

морфологічними і фонетичними (зміна закінчень, фонетичні кальки), а не лексичними замінами: для таких перетворень застосовується морфологічний аналізатор, а не словникова таблиця. Під час синтетичної генерації до кожного запиту включалось 3-5 прикладів із відповідного словника, що допомагало моделі дотримуватися лінгвістичного стилю конкретного діалекту.

1.2.5 Синтетична генерація даних

Для більшості мовних різновидів ручних паралельних даних критично мало, тому синтетичне розширення є необхідним. У роботі застосовано підхід «норма → нестандарт»: за вхідний еталон y береться літературне речення, а нестандартний варіант x генерується. Це зручно, бо якість еталону гарантована від початку.

Правилова генерація суржику. Реалізовано двофазову заміну. На першій фазі здійснюється фразова заміна: за допомогою регулярних виразів виявляються найдовші збіги із словником багатослівних суржикових конструкцій і за-

Таблиця 1.1. Словники діалектних відповідників

Мовний різновид	Записів	Формат запису
Гуцульський	7 320	Діалектна форма, стандартна форма, лема
Бойківський	14 910	Діалектна форма, стандартна форма, лема
Закарпатський	7 469	Діалектна форма, стандартна форма, лема
Суржик	570	Суржикова форма, стандартна форма

мінюються їхніми еквівалентами (наприклад, «тому що» → «потому що»). Ця фаза обробляє контекстні сполучення до розбиття тексту на окремі слова. На другій фазі виконується морфологічна заміна на рівні слів: кожне слово лематизується через бібліотеку морфологічного аналізу `ru morphology2`, і якщо словникова лема входить до списку заміन суржику, відповідне слово замінюється з урахуванням граматичних категорій - відмінку, числа і роду. Такий підхід забезпечує не просто лексичну, а морфологічно узгоджену заміну і дав 20 911 прикладів.

Генерація за допомогою ВММ. Застосована для всіх чотирьох мовних різновидів. Для кожного діалекту підготовлено окремий системний запит, що містить лінгвістичний опис типових рис, 3-5 демонстраційних пар зі словника та інструкцію щодо кількості і стилю замінів. Велика мовна модель `Mistral Small 24B` (4-бітна квантизація для локальної інференції) обробляла пакети по 5-10 речень і повертала діалектні варіанти.

Контроль якості синтетичних пар проводився після кожного пакету: видалялись дублікати (однакові вхідні або еталонні тексти), відкидались надто короткі (менше трьох слів) та надто довгі пари, перевірялась наявність діалектних слів зі словника у вхідному тексті, вибірково проводився ручний перегляд для оцінки типових артефактів генерації - змішування мов, незавершених речень, залишків форматування у відповіді.

1.2.6 Формування та структура фінального корпусу

Після генерації всіх складових дані об'єднано в єдиний корпус і розділено на три вибірки. Загальна структура наведена в таблиці 1.2.

Таблиця 1.2. Склад паралельного корпусу по мовних різновидах

Різновид	Ручні	LLM	Правилові	Разом
Гуцульський	9 853	19 841	-	29 694
Бойківський	-	29 754	-	29 754
Закарпатський	-	29 605	-	29 605
Суржик	-	29 612	20 911	50 523
Разом	9 853	108 812	20 911	139 576

Після формування загального об'єднаного набору 90% прикладів виділено у навчальну вибірку (125 615 прикладів), 10% - у валідаційну (13 756 прикладів). Тестова вибірка з 200 прикладів (по 50 на кожен мовний різновид) сформована виключно з вручну анотованих пар, що забезпечує об'єктивність оцінювання і виключає вплив якості синтетичного генератора на тестові метрики.

Ключовий принцип організації: тестова і валідаційна вибірки формуються лише з ручних пар, синтетика використовується виключно в навчальній вибірці. Це дозволяє уникнути переоцінки якості на штучних прикладах і отримувати метрики, близькі до реального застосування. Структуру директорій репозиторію і розміщення файлів корпусу показано на рис. 1.4.

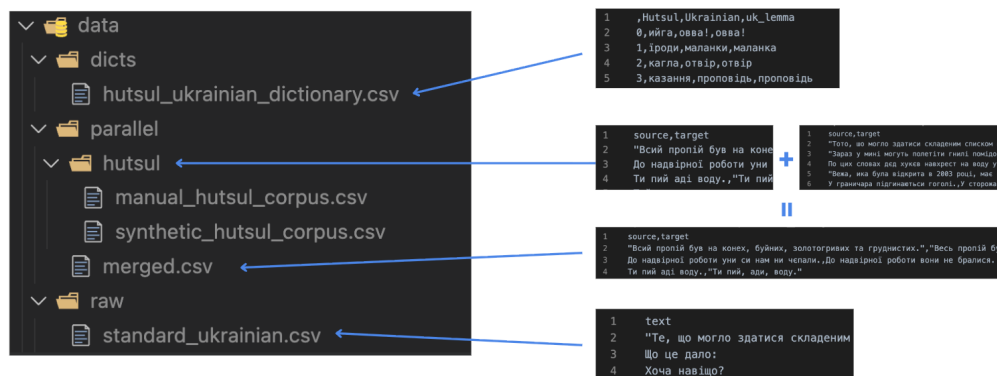


Рис. 1.4: Структура директорій проєкту: словники, ручні корпуси, синтетичні дані та фінальні вибірки для навчання, валідації і тестування.

1.3 Висновки до розділу

У розділі проаналізовано предметну галузь, розглянуто суміжні дослідження та сформовано корпус для навчання системи.

Нормалізація діалектів і суржику є задачею на межі між НМП і виправле-

нням граматичних помилок: тут присутні як систематичні лексичні заміни, так і фонетичні та морфологічні трансформації регулярного характеру. Аналіз літератури показав: для малоресурсних сценаріїв синтетичне розширення даних і донавчання ВММ є ефективними підходами, а для українських діалектів і суржику до цього часу не існувало спеціалізованих паралельних корпусів і моделей нормалізації.

Чотири досліджуваних різновиди суттєво відрізняються за природою відхилень: гуцульський і бойківський є регіональними діалектами з архаїчними фонологічними рисами, закарпатський демонструє контактний вплив суміжних мов, суржик є соціолінгвістичним явищем змішаного мовлення.

Зібрано паралельний корпус із 139 576 прикладів загалом, з яких 125 615 увійшли до навчальної вибірки. Для гуцульського діалекту використано ручний корпус (9 853 пари) і словник (7 320 записів). Для бойківського і закарпатського словники містять 14 910 і 7 469 записів відповідно. Суржиковий корпус отримано поєднанням правилowego підходу (20 911 прикладів) і генерації за допомогою ВММ (29 612). Тестові вибірки сформовано виключно з ручних пар, що забезпечує надійне оцінювання якості у наступних розділах.

2 ІНФОРМАЦІЙНЕ ТА МАТЕМАТИЧНЕ ЗАБЕЗПЕЧЕННЯ

Розділ охоплює математичну постановку задачі нормалізації, опис двох реалізованих архітектур, обґрунтування вибору моделей і конфігурацій навчання, а також метрики оцінювання. Загальну схему системи показано на рис. 2.1.

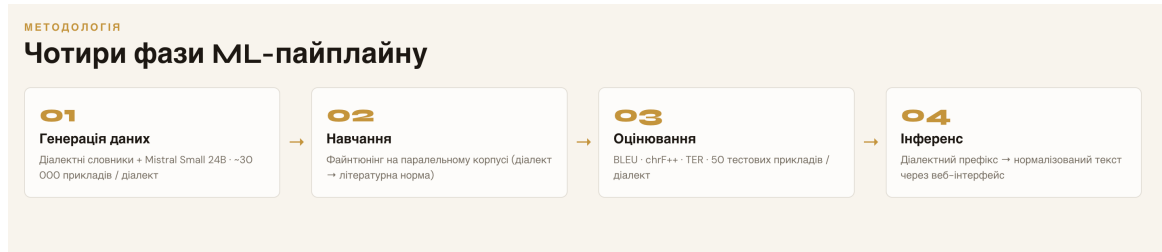


Рис. 2.1: Загальна схема системи «Лексикон»: генерація корпусу (правилова та LLM-генерація), навчання двох моделей (umt5-base і MatayLM), оцінювання на ручній тестовій вибірці та демонстраційний вебзастосунок (NiceGUI + FastAPI).

2.1 Аналіз підходів до нормалізації

2.1.1 Нормалізація як задача перетворення тексту

Нормалізацію діалектних форм і суржику розглядаємо як кероване перетворення тексту: необхідно зберегти зміст, але змінити форму відповідно до літературної норми. Такі перетворення охоплюють лексичні заміни (діалектизми), варіативну орфографію, фонетично мотивовані написання, а у суржику - ще й перемикання між мовами на рівні лексики.

Формально маємо вхідний рядок x (діалект або суржик) та бажаний вихідний рядок y (літературна українська). Мета - побудувати модель f_θ , що апроксимує відображення:

$$f_\theta : x \mapsto y \quad (2.1)$$

З погляду статистичного моделювання це еквівалентно оцінюванню умовного розподілу $p_\theta(y|x)$.

Практичні особливості задачі:

- *Часткова тотожність*: значна частина лексем у x може збігатися з y , особливо у суржику;

- *Асиметрія*: «виправляти занадто багато» ризикує спотворити зміст, «виправляти занадто мало» - не виконати нормалізацію;
- *Багатоваріантність норми*: одна діалектна форма може відповідати кільком літературним парафразам;
- *Низькоресурсність*: для більшості діалектів немає великих вручну розмічених паралельних корпусів.

2.1.2 Правиліві підходи та їх обмеження

Правилова нормалізація використовує словники відповідників, регулярні правила перетворень та евристики. Її перевага - контрольованість: якщо відомі правила заміни, можна гарантувати певну поведінку. Для словників, описаних у підрозділі 1.2.4, кожен запис задає відповідність між нестандартною і літературною формами. Попри це правилівий підхід має системні обмеження:

- складно охопити всю лексичну варіативність без великого словника;
- важко коректно обробляти контекстні значення (омонімія, полісемія);
- правила погано підходять для довгих залежностей та синтаксичних перестроїв;
- якість деградує при зміні домену (побутові повідомлення, конспекти, оголошення).

У цій роботі правиліві ресурси відіграють допоміжну роль: (а) для побудови синтетичних навчальних даних; (б) для підказок під час LLM-генерації синтетики; (в) як інструмент аналізу помилок.

2.1.3 Нейромережеві підходи: кодер-декодерна модель і декодерна ВММ

У роботі послідовно застосовано дві парадигми нейромережевого підходу.

Кодер-декодерна модель для базової перевірки на гуцульському. Переваги:

- стабільне навчання на паралельних даних;
- природна постановка: вхідний текст → нормалізований текст;
- компактний розмір (≈ 300 млн параметрів) дозволяє повне донавчання

на одному ГП.

**Декодерна ВММ з інструкційним форматом для всіх діалектів і сур-
жику.** Переваги:

- одна модель обробляє всі чотири різновиди через маркер у запиті;
- використання мовних знань моделі (≈ 4 млрд параметрів) для кращого розуміння контексту;
- природне розширення на нові задачі без зміни архітектури.

Для декодерної парадигми обрано MatayLM - відкриту україномовну ВММ на базі Gemma-3-4B, оптимізовану для інструкційного виконання задач [11.].

2.2 Постановка задачі та математична модель

2.2.1 Формальна постановка нормалізації

Нехай Σ - алфавіт символів (або множина лексем після токенізації). Вхідний текст:

$$x = (x_1, x_2, \dots, x_{|x|}), \quad x_i \in \Sigma \quad (2.2)$$

вихідний текст (літературна норма):

$$y = (y_1, y_2, \dots, y_{|y|}), \quad y_j \in \Sigma \quad (2.3)$$

Задача - знайти параметри θ , що максимізують правдоподібність навчальних пар $\{(x^{(k)}, y^{(k)})\}_{k=1}^N$:

$$\theta^* = \arg \max_{\theta} \sum_{k=1}^N \log p_{\theta}(y^{(k)} | x^{(k)}) \quad (2.4)$$

У практичній реалізації мінімізується негативний лог-правдоподібний - крос-ентропія:

$$L(\theta) = - \sum_{k=1}^N \log p_{\theta}(y^{(k)} | x^{(k)}) \quad (2.5)$$

2.2.2 Формат даних для двох архітектур

Кодер-декодерна модель. Дані зберігаються у вигляді пар «вхідний текст - еталон». Після токенізації отримуємо послідовності індексів $x = (x_1, \dots, x_T)$, $y = (y_1, \dots, y_{T'})$. Модель навчається прогнозувати y_t з урахуванням усього вхідного тексту та попередніх виходів:

$$p_\theta(y|x) = \prod_{t=1}^{T'} p_\theta(y_t|x, y_{<t}) \quad (2.6)$$

Декодерна ВММ. Задачу представляємо як інструкцію з трьома ролями: «система», «користувач» і «асистент». Вхідний текст передається у ролі «користувач», нормалізована форма - у ролі «асистент». Об'єднана послідовність лексем $z = (z_1, \dots, z_{|z|})$ включає всі три ролі. Функція втрат обчислюється лише на лексемах відповіді «асистент»:

$$L_{mask} = - \sum_{t=1}^{|z|} m_t \log p_\theta(z_t|z_{<t}) \quad (2.7)$$

де $m_t \in \{0, 1\}$ - маска (1 лише на позиціях відповіді «асистент»). Приклад шаблону інструкційного запиту наведено на рис. 2.2.

```
* System: "Ти нормалізуєш діалект/суржик до літературної української. Зберігай зміст. Не додавай зайвого."  
* User: "[ДІАЛЕКТ=ГУЦУЛЬСЬКИЙ] ...текст..."  
* Assistant: "...нормалізований текст..."
```

Рис. 2.2: Шаблон інструкційного запиту для декодерної ВММ: ролі системи, користувача та асистента з маркером діалекту.

2.2.3 Цільова функція: крос-ентропія і згладжування міток

Поелементна крос-ентропія для кодер-декодерної моделі:

$$L_{CE} = - \frac{1}{T'} \sum_{t=1}^{T'} \log p_\theta(y_t|x, y_{<t}) \quad (2.8)$$

Для umt5-base застосовано згладжування міток із коефіцієнтом $\alpha = 0.1$.

Ідея: замість жорсткого розподілу (1 для правильної лексеми, 0 для решти) навчаємо на розм'якшеному розподілі:

$$L_{smooth} = (1 - \alpha) \cdot L_{CE} + \frac{\alpha}{|V|} \sum_{v \in V} \log p_{\theta}(v|x, y_{<t}) \quad (2.9)$$

де V - словник моделі, $|V|$ - його розмір. Згладжування міток запобігає надмірній впевненості моделі та покращує узагальнення на нові приклади.

2.3 Математичні основи трансформера

2.3.1 Механізм уваги

Основою обох архітектур є трансформер із механізмом уваги. Нехай на вході є матриця представлень $H \in \mathbb{R}^{T \times d}$. Обчислюються запит, ключ і значення:

$$Q = HW_Q, \quad K = HW_K, \quad V = HW_V \quad (2.10)$$

де $W_Q, W_K, W_V \in \mathbb{R}^{d \times d_k}$ - навчані матриці проєкцій.

Увага:

$$\text{Увага}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V \quad (2.11)$$

Масштабування на $\sqrt{d_k}$ стабілізує градієнти при великих розмірностях. Багатоголова увага поєднує h незалежних голів:

$$\text{БГУ}(H) = \text{Concat}(\text{голова}_1, \dots, \text{голова}_h)W_O \quad (2.12)$$

де $\text{голова}_i = \text{Увага}(HW_{Q_i}, HW_{K_i}, HW_{V_i})$.

2.3.2 Позиційне кодування, нормалізація шарів і прямо зв'язний блок

Трансформер не має вбудованого поняття порядку - для збереження позиційної інформації до вхідних представлень додається позиційне кодування. У класичній синусоїдальній схемі:

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d}}\right), \quad PE(pos, 2i + 1) = \cos\left(\frac{pos}{10000^{2i/d}}\right) \quad (2.13)$$

де pos - позиція лексеми в послідовності, i - індекс розмірності, d - розмірність моделі. Такий підхід дозволяє розрізняти відносні відстані між лексемами без введення додаткових параметрів. У сучасних архітектурах, зокрема Gemma-3 (основа MatryLM), замість синусоїдального застосовується ротаційне позиційне кодування (RoPE), яке краще узагальнюється на довші послідовності.

Кожен підшар трансформера (увага або прямозв'язний блок) доповнюється залишковим з'єднанням і нормалізацією шарів:

$$x' = \text{LayerNorm}(x + \text{Sublayer}(x)) \quad (2.14)$$

де Sublayer - або увага, або прямозв'язний блок. Залишкове з'єднання $x + \text{Sublayer}(x)$ полегшує прямий потік градієнта через глибоку мережу і запобігає деградації навчання при великій кількості шарів.

Прямозв'язний блок (FFN), розташований після шару уваги в кожному блоці трансформера:

$$\text{FFN}(x) = \max(0, xW_1 + b_1) W_2 + b_2 \quad (2.15)$$

де $W_1 \in \mathbb{R}^{d \times d_{ff}}$, $W_2 \in \mathbb{R}^{d_{ff} \times d}$, а d_{ff} - внутрішня розмірність (зазвичай $4d$). Нелінійність ReLU ($\max(0, \cdot)$) дозволяє FFN зберігати фактичні знання про мову, тоді як механізм уваги відповідає за зв'язки між позиціями послідовності.

2.3.3 Кодер-декодерна архітектура

Кодер перетворює вхідну послідовність x у контекстні представлення $E \in \mathbb{R}^{T \times d}$ за допомогою стеку шарів: самоувага $\rightarrow$ нормалізація $\rightarrow$ прямозв'язний блок $\rightarrow$ нормалізація. Декодер генерує вихідну послідовність y , використовуючи:

- самоувагу по вже згенерованих лексемах $y_{<t}$;
- перехресну увагу до контекстних представлень E .

Умовна авторегресійна генерація:

$$p_{\theta}(y|x) = \prod_{t=1}^{T'} p_{\theta}(y_t|x, y_{<t}) \quad (2.16)$$

Базову кодер-декодерну модель umt5-base навчено спочатку на гуцульському діалекті. Графік навчання наведено на рис. 2.3, результати оцінювання - на рис. 2.4.

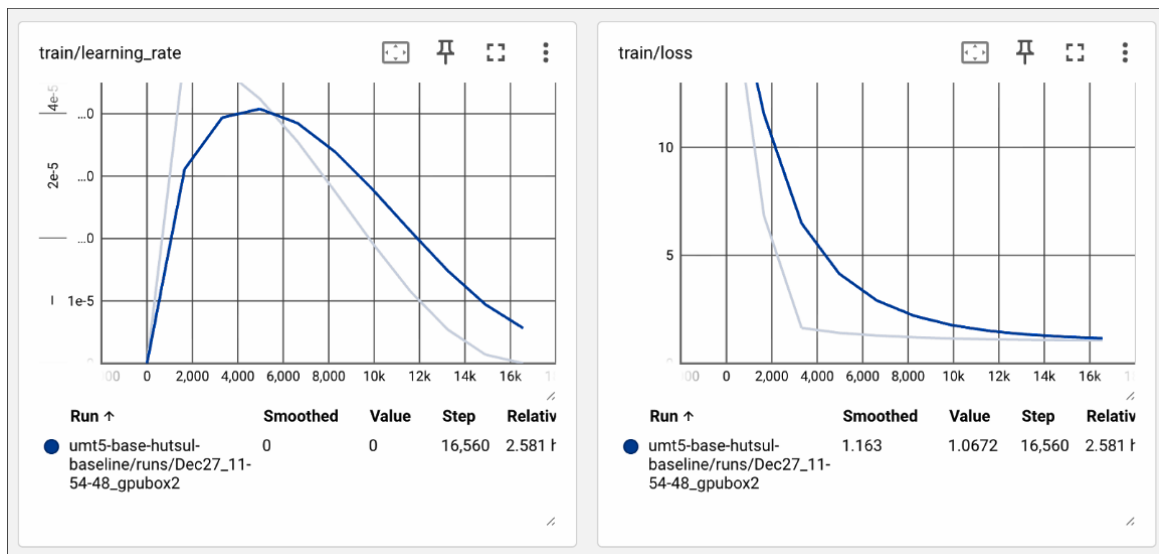


Рис. 2.3: Темп навчання та функція втрат umt5-base під час донавчання на гуцульському діалекті (перший, однодіалектний етап).

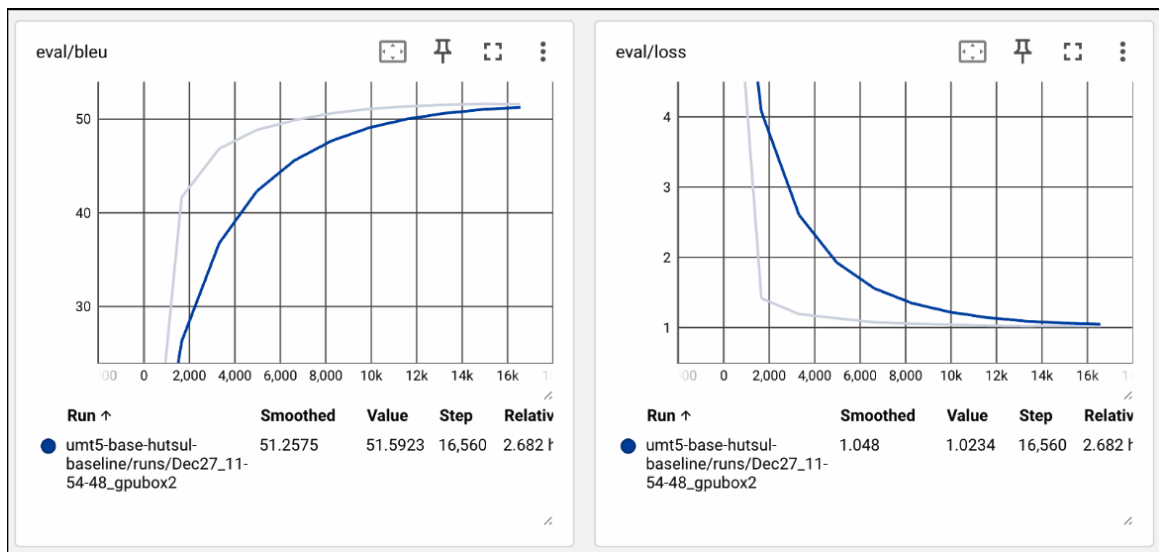


Рис. 2.4: Метрика BLEU (ліворуч) та функція втрат на валідаційній вибірці (праворуч) для гуцульського базового навчання umt5-base.

2.3.4 Авторегресійна декодерна модель

Декодерна ВММ моделює об'єднану послідовність лексем z (система + запит + відповідь) авторегресивно:

$$p_{\theta}(z) = \prod_{t=1}^{|z|} p_{\theta}(z_t | z_{<t}) \quad (2.17)$$

Усі лексеми обробляються в одному прямому проходженні, але функція втрат (2.7) оптимізується лише за лексемами відповіді «асистент». Завдяки цьому модель залишається загальною мовною моделлю, але спеціалізується на конкретній задачі нормалізації.

2.4 Вибір та обґрунтування моделей

2.4.1 Кодер-декодерна модель *umt5-base*

Для базової перевірки та кодер-декодерного підходу обрано google/umt5-base - багатомовну T5-модель (≈ 300 млн параметрів), натреновану на 100+ мовах включно з українською [3.]. Причини вибору:

1. Попереднє тренування охоплює значний обсяг українського тексту.
2. Кодер-декодерна архітектура природно підходить для нормалізації як «перекладу».
3. ≈ 300 млн параметрів дозволяє повне донавчання на одному ГП.
4. Відтворюваність: модель відкрита, розміщена у сховищі Hugging Face.

Перший етап - навчання лише на гуцульському діалекті - слугував базовою лінією. Гуцульський обрано через найбільший обсяг ручної розмітки (9 853 пари) та наявність детального словника. Мета: перевірити паралельні дані, конвеєр токенізації і навчання, а також встановити відправну точку для порівняння при переході до мультидіалектної задачі. Графік навчання (рис. 2.3) показує стабільне зниження функції втрат, BLEU на валідаційній вибірці стабілізується приблизно на рівні 51, що підтверджує коректність реалізованого конвеєра.

Другий етап - мультидіалектне навчання - об'єднує всі чотири різнови-

ди в одному корпусі (125 тис. прикладів). Спільна модель вчиться вирізняти між діалектами за допомогою системного маркера в запиті, переданому разом із текстом. Кодер-декодерна архітектура виграє від того, що кодер обробляє вхідний текст паралельно і незалежно від довжини виходу - це стабілізує навчання на різномірних парах (суржик з мінімальними замінами vs. гуцульський з фонетичними змінами).

2.4.2 Декодерна ВММ MatayLM

Для мультидіалектного підходу і суржику обрано MatayLM - декодерну ВММ на базі Gemma-3-4B (≈ 4 млрд параметрів), адаптовану для української [11.]. Причини вибору над іншими варіантами:

1. **Розмір:** 4 млрд параметрів (проти, наприклад, 12 млрд у Gemma-3-12B) дозволяє QLoRA-донавчання на одному ГП із 24 ГБ відеопам'яті.
2. **Українська мова:** модель оптимізована для задач на українській, зокрема для інструкційного виконання.
3. **Відкрита ліцензія:** результати можна публікувати і відтворювати.
4. **Інструкційна версія:** підтримує чат-шаблон (система/користувач/асистент), що спрощує навчання одночасно на чотирьох різновидах.

Інструкційний формат дозволяє явно вказувати тип завдання через маркер у ролі «система»: нормалізація гуцульського, бойківського, закарпатського або суржику. Таким чином одна модель розв'язує чотири окремі підзадачі.

2.4.3 Параметро-ефективне донавчання: LoRA та QLoRA

Повне донавчання моделі з мільярдами параметрів вимагає значних обчислювальних ресурсів. Для MatayLM застосовано LoRA - метод низькорангової адаптації (рис. 2.5).

Нехай у трансформері є матриця ваг $W \in \mathbb{R}^{d \times k}$. LoRA додає низькорангову поправку:

$$W' = W + \Delta W = W + BA \quad (2.18)$$

де $B \in \mathbb{R}^{d \times r}$, $A \in \mathbb{R}^{r \times k}$, $r \ll \min(d, k)$. Кількість тренуваних параметрів скорочується з dk до $r(d + k)$. При $r = 16$ для шару розміром 4096×4096 : замість ≈ 16.8 млн тренується лише ≈ 131 тис. параметрів.

Масштабуючий коефіцієнт регулює величину поправки:

$$s = \frac{\alpha}{r} = \frac{32}{16} = 2 \quad (2.19)$$

Цільові модулі LoRA: проєкції уваги (q_proj, k_proj, v_proj, o_proj) та прямозв'язні шари (gate_proj, up_proj, down_proj).

QLoRA доповнює LoRA 4-бітним квантуванням базової моделі у форматі NF4 (Normal Float 4) із збереженням адаптерів у bfloat16. Подвійне квантування (Double Quantization) квантує і самі коефіцієнти квантування, зменшуючи споживання ОП ще приблизно на 0.37 біт на параметр. Разом QLoRA дозволяє донавчати 4B-модель у межах 24 ГБ відеопам'яті.

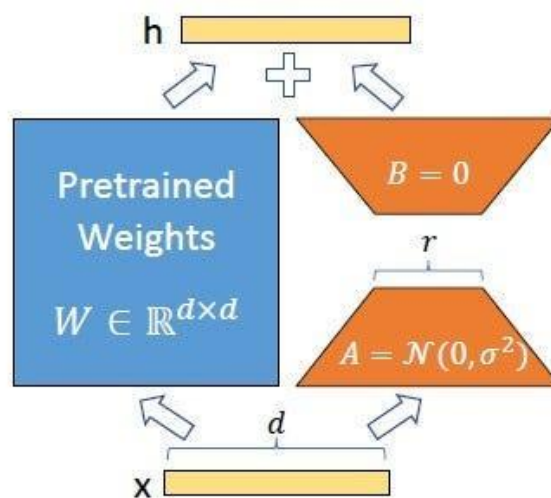


Рис. 2.5: Ілюстрація LoRA: вихідні ваги W заморожені, а низькорангова поправка BA додається у шарах уваги та прямозв'язних блоках.

2.5 Гіперпараметри та конфігурація навчання

2.5.1 Донавчання *umt5-base*

Усі гіперпараметри донавчання *umt5-base* наведено у табл. 2.1.

Вибір темпу навчання 5×10^{-5} відповідає рекомендованому діапазону для

Таблиця 2.1. Гіперпараметри донавчання umt5-base

Параметр	Значення
Базова модель	google/umt5-base
Кількість параметрів	≈ 300 млн
Кількість епох	40
Розмір пакету (накопичення: 4)	4 (ефективний: 16)
Темп навчання	5×10^{-5}
Планувальник темпу	косинусний спад
Кроки прогріву	$\min(1000, 0.1 \times N_{steps})$
Оптимізатор	AdamW 8-bit (BitsAndBytes)
Згладжування міток (α)	0.1
Спадання ваг	0.1
Числовий формат	bfloat16
Макс. довжина послідовності	256 лексем
Ранній зупин	терпіння 5 епох (за BLEU)
Заповнення	до кратного 8 лексем

повного донавчання T5-сімейства: вищі значення ризикують руйнуванням попередньо навчених представлень (catastrophic forgetting), нижчі уповільнюють збіжність на малих корпусах. Косинусний спадаючий планувальник забезпечує плавне зниження темпу навчання до нуля до кінця навчання, уникаючи різких стрибків.

Кількість епох (40) дещо більша, ніж у стандартних задачах нейронного машинного перекладу, де зазвичай достатньо 10-15 епох. Нормалізація діалектів має меншу кількість навчальних прикладів і потребує більше оновлень для якісного засвоєння. Ранній зупин із терпінням 5 епох за метрикою BLEU на валідаційній вибірці запобігає перенавчанню: якщо BLEU не покращується протягом 5 послідовних епох, навчання припиняється і зберігається найкраща контрольна точка.

8-бітний AdamW через BitsAndBytes зменшує споживання відеопам'яті оптимізатором приблизно у 4 рази порівняно зі стандартним 32-бітним AdamW, не впливаючи помітно на якість. Заповнення до кратного 8 лексем прискорює матричні операції на ГП завдяки вирівнюванню пам'яті.

2.5.2 Донавчання MatayLM (QLoRA)

Повна конфігурація донавчання MatayLM наведена у табл. 2.2.

Таблиця 2.2. Гіперпараметри донавчання MatayLM (QLoRA)

Параметр	Значення
Базова модель	MatayLM-Gemma-3-4B-IT-v1.0
Кількість параметрів	≈ 4 млрд
Кількість епох	3
Розмір пакету (накопичення: 2)	1 (ефективний: 2)
Темп навчання	2×10^{-4}
Планувальник темпу	косинусний спад
Частка прогріву	0.03
Оптимізатор	AdamW 8-bit (Unsloth)
Ранг LoRA (r)	16
Масштаб LoRA (α)	32
Відсів LoRA (dropout)	0.0
Квантування	NF4, 4-bit, подвійне
Числовий формат	bfloat16
Макс. довжина послідовності	512 лексем
Пакування послідовностей	так

Параметри генерації під час оцінювання MatayLM: температура = 0.1, top-k = 25, top-p = 1.0, максимальна кількість нових лексем = 256, штраф за повтори = 1.1.

Темп навчання 2×10^{-4} для QLoRA вищий, ніж для повного донавчання umt5-base. Це обґрунтовано тим, що LoRA-адаптери ініціалізуються з нуля (матриця B - нулями, A - малим гаусівським шумом) і не мають попередньо навченого стану, який можна пошкодити. Водночас заморожені ваги базової моделі не змінюються, тому ризик руйнування мовних знань відсутній.

Три епохи достатні для задачі нормалізації завдяки тому, що базова модель MatayLM вже добре знає українську - залишається лише сформувати стиль відповіді. Більша кількість епох ризикує переспеціалізувати модель під конкретні приклади навчальної вибірки, погіршуючи узагальнення на нових текстах.

Пакування послідовностей (packing=True) об'єднує кілька коротких при-

кладів в одну послідовність максимальної довжини 512 лексем. Це знижує частку заповнювальних лексем (padding) у пакеті та підвищує ефективне використання ГП: замість навчання на переважно порожніх пакетах модель отримує більше фактичного навчального сигналу за той самий час.

2.6 Метрики оцінювання

2.6.1 BLEU

BLEU (Bilingual Evaluation Understudy) оцінює схожість згенерованого тексту з еталоном через модифіковану n-грамну точність [16.]:

$$\text{BLEU} = BP \cdot \exp\left(\sum_{n=1}^N w_n \log p_n\right) \quad (2.20)$$

де p_n - модифікована точність n-грам, $w_n = 1/N$ - рівні ваги, BP - штраф за стислість:

$$BP = \begin{cases} 1, & |\hat{y}| > |y| \\ \exp\left(1 - \frac{|y|}{|\hat{y}|}\right), & |\hat{y}| \leq |y| \end{cases} \quad (2.21)$$

де $|\hat{y}|$ - довжина згенерованого тексту, $|y|$ - довжина еталону.

У роботі обчислюється `corpus_bleu` із бібліотеки `SacreBLEU`. Значення BLEU є критерієм збереження найкращої контрольної точки під час навчання `umt5-base`.

2.6.2 chrF++

`chrF++` розширює метрику `chrF`, додаючи до символьних n-грам словесні (параметр `word_order = 2`). Базова F-міра:

$$\text{chrF} = \frac{(1 + \beta^2) \cdot \text{chrP} \cdot \text{chrR}}{\beta^2 \cdot \text{chrP} + \text{chrR}} \quad (2.22)$$

де `chrP` - символьна точність, `chrR` - символьна повнота, $\beta = 2$ (більший акцент на повноту, ніж на точність).

Переваги `chrF++` для задачі нормалізації:

- нечутлива до токенизації - оцінює безпосередньо символи;
- краще відображає морфологічну схожість для мов із розгалуженою флексійною системою;
- стабільніша за BLEU на коротких виходах, де n-грамні підрахунки нестійкі.

У роботі chrF++ є основною порівняльною метрикою. Обчислюється як `corpus_chrf(word_order = 2)` у SacreBLEU.

2.6.3 TER

TER (Translation Edit Rate) вимірює кількість редагувань, необхідних для перетворення згенерованого тексту на еталон:

$$\text{TER} = \frac{\text{кількість редагувань}}{|\text{еталон}|} \quad (2.23)$$

Типи редагувань: вставка, видалення, заміна та зсув фрагмента. На відміну від BLEU і chrF++, менше значення TER означає кращу якість. Обчислюється як `corpus_ter` у SacreBLEU.

2.6.4 Взаємодоповнюваність метрик

Три метрики охоплюють різні аспекти якості і доповнюють одна одну:

- BLEU акцентує на точності n-грамних збігів слів - добре відображає, чи вибирає модель правильну лексику;
- chrF++ оцінює морфологічну близькість через символні n-грами - важливо для українського, де одне слово може мати кілька відмінкових форм;
- TER показує мінімальний обсяг правок для отримання еталону - інтуїтивно зрозуміла метрика для оцінки «скільки ще треба виправити».

Для задачі нормалізації до близькоспорідненої літературної норми типові очікувані діапазони: chrF++ > 90 для суржику (мінімальні лексичні відмінності) та суттєво нижчі значення для діалектів із більшою фонетичною і морфологічною відстанню від норми, таких як закарпатський. Низький TER при невисо-

кому BLEU може сигналізувати про правильні слова у неправильному порядку або часткові збіги.

2.7 Стратегії декодування

Задача декодування - знайти найбільш імовірну вихідну послідовність:

$$\hat{y} = \arg \max_y \log p_{\theta}(y|x) \quad (2.24)$$

Жадібний пошук: на кожному кроці вибирається лексема з найвищою ймовірністю:

$$\hat{y}_t = \arg \max_{y_t} p_{\theta}(y_t|x, \hat{y}_{<t}) \quad (2.25)$$

Найшвидший варіант, але може «зациклюватись» або пропустити глобально кращу послідовність.

Пошук з променем (beam search, B - розмір променя): підтримує B найкращих часткових гіпотез одночасно. Для umt5-base під час оцінювання $B = 4$. Краща якість за рахунок перебору, ціна - лінійне збільшення обчислень.

Семплювання з температурою (для MatayLM): лексема вибирається із розподілу, загостреного температурою τ :

$$p'_{\theta}(y_t|x, \hat{y}_{<t}) \propto \exp\left(\frac{\log p_{\theta}(y_t|x, \hat{y}_{<t})}{\tau}\right) \quad (2.26)$$

При $\tau = 0.1$ розподіл загострюється і наближається до жадібного пошуку, зберігаючи невелику варіативність. Додатково застосовується top-k фільтрування ($k = 25$): решту лексем відкидають.

Відмінність стратегій: umt5-base генерує детермінований вихід (пошук з променем), MatayLM - з незначним різноманіттям через семплювання.

2.8 Балансування даних

2.8.1 Дисбаланс діалектів та принципи семплювання

Мультидіалектний корпус нерівномірний: суржик охоплює ≈ 50 тис. прикладів, тоді як гуцульський вручну - менше 10 тис. При навчанні на об'єднаному

корпусі оптимізація зміщується у бік найбільшого підкорпусу. Для вирівнювання вводяться ваги семплювання:

$$P(\text{діалект} = i) \propto n_i^\gamma \quad (2.27)$$

де n_i - кількість прикладів діалекту i , а $\gamma \in [0, 1]$ контролює ступінь вирівнювання ($\gamma = 0$ - рівномірний, $\gamma = 1$ - пропорційний до обсягу).

2.8.2 Формат інструкцій для мультидіалектності

Для декодерної BMM використовується єдиний формат із маркером діалекту в системній ролі:

- нормалізація гуцульського (маркер «HUTSUL»);
- нормалізація бойківського (маркер «BOIKO»);
- нормалізація закарпатського (маркер «TRANSCARPATHIAN»);
- нормалізація суржиків (маркер «SURZHUK»).

Таким чином система розв'язує умовну задачу $p_\theta(y|x, t)$, де t - тип нормалізації, закодований у вхідному запиті.

2.9 Висновки до розділу

У розділі наведено математичну постановку задачі нормалізації як умовної генерації $p_\theta(y|x)$. Кодер-декодерна модель `umt5-base` [3.] пройшла повне донавчання за 40 епох із кроком навчання 5×10^{-5} , згладжуванням міток $\alpha = 0.1$ (формула 2.9) та раннім зупином. Декодерна BMM `MatayLM` [11.] навчена через QLoRA ($r = 16$, $\alpha = 32$, 4-bit NF4) за 3 епохи з кроком 2×10^{-4} . Описано математичні основи: механізм уваги трансформера (формула 2.11), цільові функції (формули 2.8–2.9), стратегії декодування (жадібний пошук та пошук з променем) і метрики BLEU [16.], chrF++ та TER (формули 2.20–2.23).

3 ПРОГРАМНЕ ТА ТЕХНІЧНЕ ЗАБЕЗПЕЧЕННЯ

3.1 Технологічний стек

Систему реалізовано на Python 3.11 з фіксованими версіями залежностей у файлі `pyproject.toml`. Керування залежностями виконується через інструмент `uv`, а всі основні операції автоматизовано через `Makefile`: встановлення середовища (`install`), підготовка даних (`prepare-data`), генерація корпусів (`generate-*`), навчання моделей (`train-*`), оцінювання (`evaluate-*`), запуск застосунку (`demo`) та компіляція документації (`pdf`).

Перевага такого підходу - відтворюваність: будь-який дослідник може клонувати репозиторій і відтворити всі результати однією командою `make all`, не виконуючи ручних кроків. Зокрема, `make prepare-data` завантажує вихідні речення, запускає правилу та LLM-генерацію і формує фінальний навчальний корпус повністю автоматично. Цей підхід до автоматизації знижує ризик помилок відтворення, притаманних ручним інструкціям.

Основні бібліотеки, задіяні у системі, наведено в таблиці 3.1.

Бібліотека `Unsloth` є ключовою для прискорення QLoRA-навчання. Вона перезаписує критичні операції (функція уваги, обчислення `softmax` у `cross-entropy`) нативними ядрами `CUDA` через `Triton-JIT`-компіляцію, що дає зниження споживання відеопам'яті на 30-40% порівняно зі стандартною реалізацією `Transformers`. Додатково `Unsloth` підтримує асинхронне збереження контрольних точок у фоновому потоці, не призупиняючи навчання. Бібліотека `rumorphy2` забезпечує морфологічний аналіз для правилкової генерації суржику: без неї підтримка відмінкових форм заміників була б неможливою без великої вручну складеної таблиці.

Крім програмних залежностей, важливу роль відіграє технічне забезпечення. Різні етапи роботи системи ставлять різні вимоги до обчислювальних ресурсів (табл. 3.2).

Фактично у роботі використано сервер з `NVIDIA A100 (80 GB VRAM)`,

Таблиця 3.1. Основні бібліотеки системи

Бібліотека	Версія	Призначення
PyTorch	2.6.0	Основний фреймворк для навчання нейронних мереж
Transformers	$\geq 4.57.1$	Трансформерні архітектури, токенизатори, конвеєри
PEFT	$\geq 0.15.0$	LoRA/QLoRA параметро-ефективне донавчання
TRL	0.18.2	SFTTrainer для навчання з інструкціями
Unsloth	2026.1.4	Прискорення QLoRA (2-3 рази): зли-та увага, оптимізований autograd
BitsAndBytes	$\geq 0.45.0$	4- та 8-бітне квантування вагових матриць
Accelerate	$\geq 1.12.0$	Розподілені обчислення та змішана точність
SacreBLEU	$\geq 2.5.1$	Обчислення BLEU, chrF++ та TER
Datasets	3.4.1	Завантаження та обробка наборів даних
NiceGUI	$\geq 1.4.0$	Вебзастосунок (Python-native UI над FastAPI)
PyAV (av)	$\geq 14.0.0$	Декодування аудіо WebM/OGG/WAV до 16 кГц
py morphology2	актуальна	Морфологічний аналіз і відмінювання слів
Typeer	$\geq 0.9.0$	Інтерфейси командного рядка скриптів

128 ГБ оперативної пам'яті та NVMe-диском обсягом 1 ТБ. Такі ресурси дозволили виконати всі етапи без обмежень за пам'яттю та сховищем. Найбільш ресурсомістким є етап генерації синтетики: модель Mistral-Small-24B потребує понад 16 ГБ VRAM навіть у 4-бітному режимі, а генерація 30 000 прикладів на діалект займає від 12 до 24 годин на NVIDIA A100.

3.2 Конвеєр генерації корпусу

3.2.1 Правилова генерація суржику

Для суржику реалізовано клас SurzhykErrorifier, що перетворює нормативні українські речення у суржик без залучення зовнішніх ВММ. Такий підхід

Таблиця 3.2. Вимоги до технічного забезпечення за етапами

Компонент	Генерація синтетики	Навчання umt5	QLoRA MamayLM	Застосунок
ЦП	8+ ядер	8+ ядер	8+ ядер	4+ ядра
ОП	32 ГБ	16 ГБ	32 ГБ	8 ГБ
ГП (VRAM)	24 ГБ	8 ГБ	24 ГБ	4 ГБ
Сховище	50 ГБ	10 ГБ	20 ГБ	5 ГБ

дозволив отримати додатково 20 911 прикладів за рахунок детермінованих правил, рівномірно покриваючи типові лексичні підстановки. Правильний підхід забезпечує стовідсоткову відтворюваність: однакове речення завжди дає однаковий суржиковий варіант.

Алгоритм складається з двох послідовних фаз.

Фаза 1: фразова заміна. Клас будує лексикон фраз: усі записи словника, що складаються з кількох слів, упорядковуються за спаданням довжини у лексемах. Для кожного вхідного речення алгоритм застосовує ці фрази як регулярні вирази (з урахуванням меж слів \b) від найдовшої до найкоротшої, підставляючи відповідний суржиковий варіант при першому збігу. Пріоритет найдовшого збігу запобігає частковим замінам: наприклад, фраза «на самому ділі» замінюється цілком, а не лише слово «самом». Усі заміщені позиції у реченні позначаються як вже оброблені і не передаються до фази 2.

Фаза 2: пословна морфологічна заміна. Для кожного слова, ще не заміненого у фазі 1, виконується лематизація через `ru morphology2`. Якщо лема знайдена у лемному індексі словника, береться суржиковий відповідник. Далі `ru morphology2` відмінює його у потрібну граматичну форму, збігаючи граммами оригінального слова (рід, число, відмінок, особа, час). Це гарантує граматичну узгодженість результату: якщо вхід містить слово у родовому відмінку множини, суржиковий заміник також опиняється у тій самій формі. Якщо потрібна граматична форма відсутня у парадигмі замінника (наприклад, незмінювані слова), система повертає початкову форму без помилки.

Речення відкидається, якщо жодної заміни не відбулося. Відношення ефе-

ктивності: з усього доступного корпусу нормативних речень правилловий підхід дав позитивний результат (хоча б одну заміну) приблизно для 40% речень, що й склало 20 911 прикладів суржику.

3.2.2 LLM-генерація для діалектів

Для генерації бойківського, закарпатського та гуцульського діалектів, а також додаткового суржику, реалізовано клас `DialectCorpusGenerator`. Він складається з двох взаємодіючих компонентів: `DialectPromptBuilder` та `LocalModelClient`.

DialectPromptBuilder відповідає за формування інструкційного запиту до моделі. Для кожного діалекту завантажується відповідний файл правил з теки `prompts/` (337-568 рядків). Щоб не включати до контексту весь словник (що може перевищувати ліміт лексем моделі), застосовується відбір найрелевантніших статей. Обчислюється TF-IDF косинусна схожість між вхідним реченням та кожним записом словника, і до запиту включаються лише $k = 50$ найближчих статей. Таким чином кожне речення отримує сфокусовану лексичну підказку, адаптовану до його конкретної семантики: речення про транспорт отримує транспортну лексику, а речення про побут - побутову. Шаблон запиту наведено на рис. 2.2 (підрозд. 2.2.2).

Відібрані словникові статті замінюють плейсхолдер `{dictionary}` у системному запиті. Запит користувача містить пакет з 5 вхідних речень, пронумерованих списком. Модель повертає відповідний список з 5 перекладів у форматі JSON-масиву рядків.

LocalModelClient виконує інференцію на локально розгорнутій моделі `Mistral-Small-24B-Instruct-2501` з 4-бітним квантуванням NF4 у режимі подвійного квантування. Локальний запуск обрано замість хмарного API з двох причин: по-перше, це усуває залежність від зовнішніх сервісів і ліміти запитів; по-друге, забезпечує повний контроль над параметрами генерації (`temperature=0.7`, `max_tokens=2048`). Пакетна обробка по 5 речень за запит дозволяє моделі розглядати речення у спільному контексті, що підвищує стилістичну узгодженість

перекладів у межах одного пакету. При обробці ізольованих речень модель може обрати різний стиль для схожих фраз; спільний контекст 5 речень знижує таку варіативність.

Клас `DialectCorpusGenerator` реалізує чотирирівневий контроль якості:

- **Подвійний розбір відповіді:** спершу спроба декодувати JSON-масив рядків; якщо не вдається - текстовий запасний варіант (розбиття за номерованими рядками).
- **Валідація:** відсіюються пари, де вихід коротший за третину входу (ознака незавершеності відповіді), або де присутні залишки JSON-синтаксису, або де кількість виходів не збігається з кількістю входів.
- **Дедуплікація:** хеш нормалізованого рядка (нижній регістр, без зайвих пробілів) зберігається у множині; дублікати не додаються до корпусу незалежно від сеансу генерації.
- **JSONL-журнал:** кожен прийнятий запис зберігається з полем `source`, що містить ідентифікатор діалекту та номер пакету. Це дозволяє відстежити походження будь-якого прикладу у корпусі.

Ліміт генерації встановлено на рівні 30 000 прикладів на діалект. Фактично отримано: 19 841 для гуцульського (ліміт не досягнуто через менший вихідний корпус речень), по 29 754 для бойківського та 29 612 для суржику. Повторні запуски при збоях безпечні завдяки дедуплікації: вже згенеровані приклади просто пропускаються.

3.2.3 Підготовка корпусу до навчання

Фінальний корпус формується скриптом `prepare_multidialect_data.py`. Він об'єднує підкорпуси всіх діалектів, перемішує їх і ділить на навчальну і валідаційну вибірки у пропорції 90/10. Тестова вибірка формується цілком окремо: вона містить виключно ручні анотації (по 50 прикладів на діалект, 200 разом) і жодним чином не перетинається з навчальними даними. Такий поділ гарантує чисте оцінювання: модель не могла «бачити» тестові речення у жодній формі під час навчання.

Розподіл підсумкового корпусу наведено в таблиці 3.3.

Таблиця 3.3. Розподіл корпусу за різновидами

Різнovid	Ручні	LLM	Прав.	Навч.	Валід.	Тест
Гуцульський	9 853	19 841	-	26 725	2 969	50
Бойківський	-	29 754	-	26 778	2 976	50
Закарпатський	-	29 605	-	26 644	2 961	50
Суржик	-	29 612	20 911	45 471	5 052	50
Разом	9 853	108 812	20 911	125 615	13 756	200

Підкорпус суржику є найбільшим завдяки двом незалежним підходам до генерації: LLM-синтетика і правилова генерація не перетинаються за методом, тож їх об'єднання дає різноманітніший корпус. Підкорпус гуцульського діалекту вирізняється наявністю 9 853 ручних пар, що позитивно вплинуло на якість навчання. Підкорпуси бойківського та закарпатського діалектів сформовані виключно з LLM-синтетики через відсутність публічних паралельних корпусів для цих різновидів.

3.3 Навчання моделей

3.3.1 Донавчання *umt5-base*

Навчання кодер-декодерної моделі *umt5-base* [3.] реалізовано через клас *Seq2SeqTrainer* з бібліотеки *Transformers*. Процес складається з шести основних кроків.

Крок 1: завантаження та підготовка даних. CSV-файли з навчальною і валідаційною вибірками завантажуються в об'єкти *HuggingFace Dataset* з полями *source* (діалектний текст) та *target* (нормативна відповідь). Для мультидіалектної моделі поле *source* містить префікс, наприклад: «Переклади з гуцульської: *вхідний текст*». Префіксний підхід дозволяє єдиній моделі обслуговувати всі чотири різновиди без окремих ваг.

Крок 2: токенизація. Вхідні та цільові тексти кодуються через *AutoTokenizer* (*SentencePiece*, словник 250 112 лексем). Максимальна довжина входу і виходу обмежена 256 лексемами - цього достатньо для всіх речень у корпусі, оскільки середня довжина речення не перевищує 30 слів.

Крок 3: динамічне доповнення. Замість фіксованого доповнення до 256 лексем застосовується `DataCollatorForSeq2Seq` з параметром `pad_to_multiple_of=8`: кожен пакет доповнюється лише до кратної 8 довжини найдовшого речення у пакеті. Це мінімізує кількість паддінг-лексем і підвищує ефективність матричних операцій на CUDA, оскільки ядра тензорних обчислень оптимізовані для розмірів, кратних 8 або 16.

Крок 4: навчання. Використовується `Seq2SeqTrainer` з гіперпараметрами, описаними у підрозділі 2.5 (табл. 2.1). Функція втрат - перехресна ентропія зі згладжуванням міток $\alpha = 0.1$ (формула 2.9), що запобігає надмірній впевненості моделі у окремих лексемах і покращує узагальнення.

Крок 5: моніторинг і рання зупинка. Після кожної епохи обчислюється BLEU на валідаційній вибірці через функцію `compute_metrics`: декодуються передбачення (ігноруються спеціальні ID доповнення —100), обчислюється BLEU через `sacrebleu` з детоксифікацією. Якщо BLEU не покращується упродовж 5 епох поспіль, навчання зупиняється достроково. Зберігаються три останні контрольні точки; по завершенні автоматично завантажується найкраща за BLEU.

Крок 6: збереження. Фінальна модель і токенизатор зберігаються у каталог `models/umt5-base-multidialect/final_model`, звідки їх завантажує застосунок «Лексикон» у режимі реального часу.

Графіки мультидіалектного навчання (суржик + 3 діалекти) наведено на рис. 3.1 та 3.2. Пікове значення BLEU 71.7 досягнуто на кроці 314 040.

3.3.2 Донавчання MatayLM

Навчання декодерної BMM MatayLM [11.] (Gemma-3-4B) реалізовано через `SFTTrainer` з бібліотеки `TRL` з прискоренням `Unsloth`. Ключова особливість порівняно з кодер-декодерним підходом - необхідність явного форматування прикладів у чат-шаблон і маскування функції втрат.

Крок 1: завантаження та квантування. Базова модель завантажується у режимі 4-бітного квантування NF4 з подвійним квантуванням (`BitsAndBytesConfig`). Подвійне квантування означає, що коефіцієнти кванту-

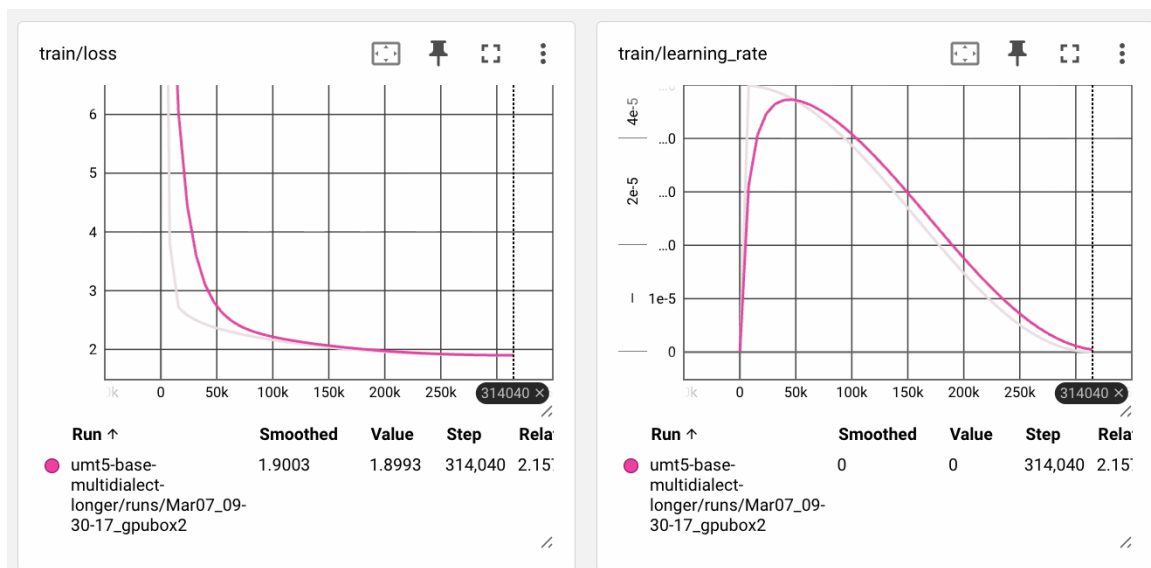


Рис. 3.1: Функція втрат та темп навчання umt5-base під час мультідіалектного донавчання (суржик + 3 діалекти, 314 040 кроків).

вання самих ваг теж квантуються, що дає додаткову економію близько 0.4 біту на параметр. Якщо доступна бібліотека Unsloth - використовується її оптимізований завантажувач; інакше модель завантажується через стандартний Transformers разом з PEFT.

Крок 2: форматування у чат-шаблон. Кожен навчальний приклад форматується у два повідомлення: користувача («Переклади з {діалект}: {вхідний текст}») та асистента («{нормативний текст}»). Функція `apply_chat_template` перетворює ці пари у послідовність лексем відповідно до формату Gemma-3 (спеціальні маркери початку і кінця ходу). Префіксний метод обумовлення на діалект зберігається з umt5-підходу для сумісності.

Крок 3: пакування. SFTTrainer з параметром `packing=True` об'єднує короткі приклади у пакети до 512 лексем. Замість того щоб доповнювати кожен приклад до 512 лексем паддінгом (що давало б велику частку «порожніх» лексем), пакування заповнює кожен пакет реальним текстом майже повністю. Це суттєво підвищує завантаженість ГП і прискорює навчання: у практиці пакування дало приріст пропускну здатності близько 40% порівняно з непакованим режимом на тому самому обладнанні.

Крок 4: маскування функції втрат. Градієнт обчислюється лише за ле-

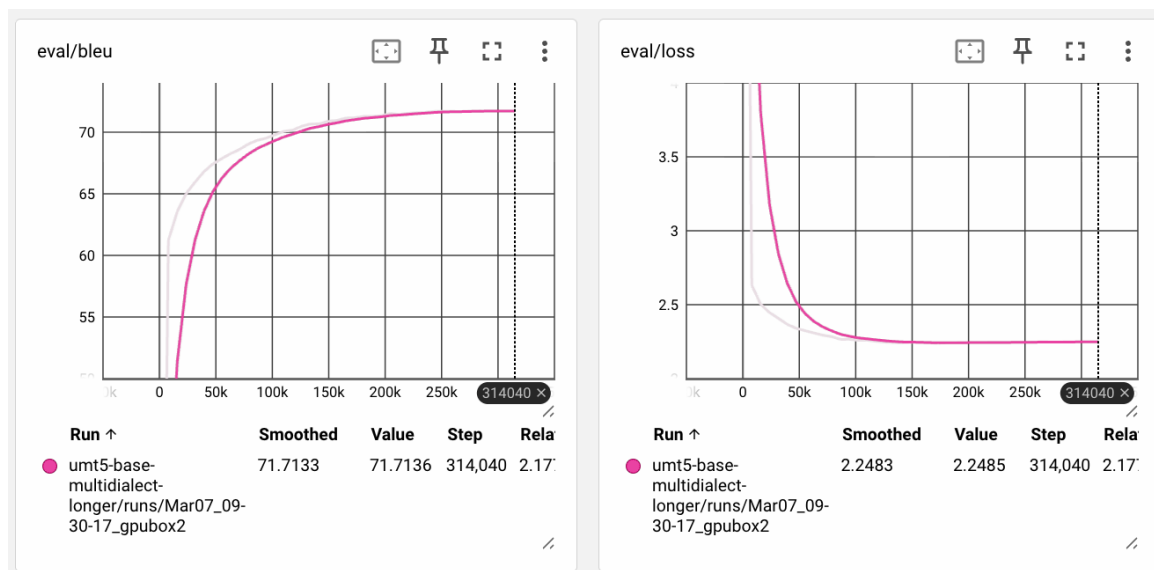


Рис. 3.2: Динаміка BLEU та функція втрат на валідаційній вибірці під час мультидіалектного донавчання *umt5-base*.

ксемами відповіді асистента. Лексеми системного запиту та запиту користувача замінюються на -100 (ігнорований індекс у стандартній функції перехресної ентропії PyTorch), тому модель навчається генерувати нормативну відповідь, а не відтворювати вхідний запит.

Крок 5: LoRA-адаптери. QLoRA-адаптери ($r = 16$, $\alpha = 32$, dropout=0.0) застосовуються до всіх проекційних матриць уваги та FFN-шарів: `q_proj`, `k_proj`, `v_proj`, `o_proj`, `gate_proj`, `up_proj`, `down_proj`. Заморожені ваги базової моделі зберігаються у 4-бітному форматі і не займають оперативну пам'ять ГП в повній точності. Ефективна кількість навчальних параметрів - близько 13 мільйонів, що становить менш ніж 0.4% від загального обсягу моделі у 4 мільярди параметрів.

Крок 6: контрольні точки та завершення. Unsloth забезпечує асинхронне збереження контрольних точок у фоновому потоці без паузи у навчанні. Після завершення можливе злиття адаптерів у базову модель (`merge_and_unload`) для ефективного розгортання без накладних витрат PEFT під час інференції.

Частково захоплені криві навчання MatayLM наведено на рис. 3.3. TensorBoard активовано приблизно з кроку 25 000, тому початковий спад функції втрат на графіку відсутній.

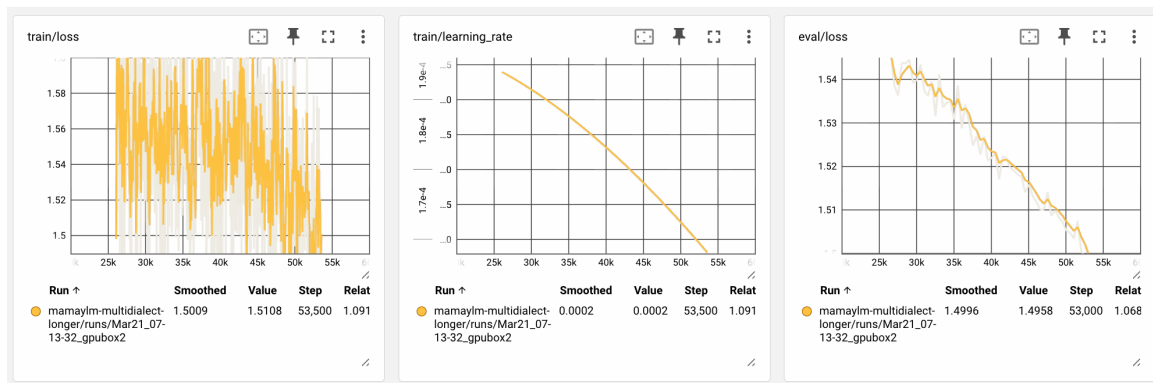


Рис. 3.3: Функція втрат та темп навчання MamayLM (QLoRA) з кроку 25 000 до 53 500 (TensorBoard увімкнено з проміжного кроку навчання).

3.4 Оцінювання та аналіз результатів

3.4.1 Методологія оцінювання

Тестова вибірка містить 200 прикладів: по 50 на кожен з чотирьох різновидів. Усі приклади зібрані вручну і жодним чином не перетиналися з навчальними даними. Ручна анотація тестових прикладів є принципово важливою: якби тестова вибірка містила LLM-синтетику, результати відображали б якість того самого генератора, яким навчалися моделі, а не реальну лінгвістичну точність.

Нуль-шотовими базовими лініями слугують три комерційні BMM, доступні через OpenRouter API: GPT-4o [16.], Gemini 2.0 Flash та Mistral Large. Їх запитували з тим самим системним запитом («Переклади з {діалект} до літературної української мови»), але без жодних прикладів у контексті - нуль-шотовий режим. Це дозволяє оцінити, наскільки спеціалізоване донавчання перевершує загальні BMM, що не мали доступу до наших навчальних даних.

Для оцінювання кодер-декодерних моделей реалізовано скрипт `evaluate_encoder_decoder.py`, для декодерних BMM - `evaluate_decoder_only.py`. Обидва зберігають результати у `results.json` з розбивкою показників за кожним з чотирьох різновидів. Обчислюються три метрики, описані у підрозділі 2.6: BLEU (формула 2.20), chrF++ та TER (формула 2.23).

3.4.2 Результати кодер-декодерної моделі *umt5-base*

Зведені результати *umt5-base* по всіх різновидах наведено на рис. 3.4.

UMT5-base (finetuned, encoder-decoder)

Tasks	bleu	chrf++	ter
boiko_translation	29.12	54.78	49.37
hutsul_translation	74.68	86.90	12.53
surzhyk_translation	89.42	93.58	5.98
transcarpathian_translation	6.20	27.45	82.21

Рис. 3.4: Результати umt5-base на тестовій вибірці за різновидами (BLEU, chrF++, TER).

Найкращий результат - суржик (chrF++ = 93.58). Це пояснюється найбільшим навчальним підкорпусом (понад 50 000 пар двох типів генерації) та лексичною близькістю суржику до нормативної мови: більшість слів збігаються, відрізняються лише окремі лексеми. Гуцульський діалект демонструє chrF++ = 86.90, що зумовлено найбільшою кількістю ручних анотацій (9 853 пари). Бойківський результат нижчий (chrF++ = 54.78): цей діалект має складніші синтаксичні і фонетичні відмінності від норми, а кодер-декодерна архітектура може недостатньо вловлювати їх при суто лексичних підстановках. Закарпатський дає найслабший результат (chrF++ = 27.45): суттєва лінгвістична відстань від літературної норми і відсутність ручних прикладів у навчанні.

3.4.3 Результати декодерної BMM MamayLM

Результати MamayLM наведено на рис. 3.5.

MamayLM (finetuned, decoder-only)

Tasks	bleu	chrf++	ter
boiko_translation	51.81	66.82	35.26
hutsul_translation	64.84	74.02	12.53
surzhyk_translation	78.48	91.22	11.96
transcarpathian_translation	3.82	20.93	89.69

Рис. 3.5: Результати MamayLM на тестовій вибірці за різновидами (BLEU, chrF++, TER).

MamayLM демонструє сильний результат на суржику (chrF++ = 91.22) і на бойківському (chrF++ = 66.82), де вона перевершує umt5-base. На гуцульському

результат нижчий (74.02 проти 86.90 у umt5), що може пояснюватися меншою кількістю навчальних кроків (3 епохи проти 40) і природою декодерних ВММ: такі моделі краще виявляють семантичні паттерни, але гірше «запам'ятовують» точні лексичні відповідності, важливі для символічних метрик. Закарпатський залишається найскладнішим і для MamayLM (chrF++ = 20.93).

3.4.4 Порівняння з нуль-шотовими базовими лініями

Результати нуль-шотових базових ліній наведено на рисунках 3.6-3.9.

Tasks	Filter	n-shot	Metric	Value	Stderr	Stderr_CLT
boiko_translation	none	0	chrF_score	0.7395±	N/A	0.0199
hutsul_translation	none	0	chrF_score	0.6489±	N/A	0.0255
surzhyk_translation	none	0	chrF_score	0.8897±	N/A	0.0176
transcarpathian_translation	none	0	chrF_score	0.6539±	N/A	0.0218

Рис. 3.6: Результати нуль-шотового GPT-4o за різновидами.

Tasks	Filter	n-shot	Metric	Value	Stderr	Stderr_CLT
boiko_translation	none	0	chrF_score	0.7457±	N/A	0.0199
hutsul_translation	none	0	chrF_score	0.6534±	N/A	0.0245
surzhyk_translation	none	0	chrF_score	0.8667±	N/A	0.0193
transcarpathian_translation	none	0	chrF_score	0.6816±	N/A	0.0210

Рис. 3.7: Результати нуль-шотового Gemini 2.0 Flash за різновидами.

Tasks	Filter	n-shot	Metric	Value	Stderr	Stderr_CLT
boiko_translation	none	0	chrF_score	0.7091±	N/A	0.0225
hutsul_translation	none	0	chrF_score	0.6318±	N/A	0.0233
surzhyk_translation	none	0	chrF_score	0.8579±	N/A	0.0209
transcarpathian_translation	none	0	chrF_score	0.6365±	N/A	0.0250

Рис. 3.8: Результати нуль-шотового Mistral Large за різновидами.

Зведену таблицю chrF++ для всіх моделей і різновидів наведено в таблиці 3.4.

3.4.5 Аналіз результатів

Аналіз показників дозволяє зробити кілька спостережень.

Суржик. Обидві донавчені моделі перевершують усі нуль-шотові базові лінії, включно з найсильнішою з них - Gemini 2.0 Flash (93.58 і 91.22 проти 86.67). Основна причина - великий корпус (понад 50 000 пар) та лексична

MamayLM (baseline, before finetuning)

Tasks	bleu	chrF++	ter
boiko_translation	21.77	58.73	56.42
hutsul_translation	5.06	16.73	50.13
surzhyk_translation	49.07	65.58	35.89
transcarpathian_translation	10.43	27.90	82.21

Рис. 3.9: Нуль-шотові результати MamayLM (до донавчання) за різновидами.

Таблиця 3.4. Порівняння моделей за метрикою chrF++ на тестовій вибірці

Модель	Суржик	Гуцульський	Бойківський	Закарпатський
umt5-base (наш)	93.58	86.90	54.78	27.45
MamayLM (наш)	91.22	74.02	66.82	20.93
GPT-4o	88.97	64.89	-	65.39
Gemini 2.0 Flash	86.67	65.34	74.57	68.16
Mistral Large	85.79	63.18	70.91	63.65
MamayLM (до донавч.)	65.58	16.73	58.73	27.90

близькість суржика до норми: більшість слів ідентична, змінюється лише 20-30% лексики. Показово, що нуль-шотовий MamayLM до донавчання дав лише 65.58 - тобто донавчання підняло ту саму модель на 25.64 пункти. Це свідчить, що основний внесок дають саме спеціалізовані навчальні дані, а не загальна потужність архітектури.

Гуцульський. umt5-base (86.90) і MamayLM (74.02) суттєво перевищують усі нуль-шотові моделі, найближча з яких - Gemini (65.34). Вирішальним фактором став найбільший ручний підкорпус (9 853 пари): донавчені моделі вивчили специфічну фонетику і лексику гуцульського, яку комерційні BMM знають лише з передтренувального корпусу. MamayLM до донавчання показав лише 16.73 на гуцульському - це найбільший розрив між базовою і донавченою версією серед усіх різновидів, що підтверджує ефективність ручних анотацій.

Бойківський. Тут ситуація інша: найкращий результат у Gemini 2.0 Flash (74.57), далі Mistral Large (70.91), і лише потім MamayLM (66.82) і umt5-base

(54.78). Наші донавчені моделі не виходять на перше місце. Причина - корпус бойківського складається виключно з LLM-синтетики: якщо синтетика неточно відображає реальні властивості діалекту, модель навчається неправильним патернам. Комерційні ВММ, навчені на великих корпусах, у цьому випадку краще узагальнюють.

Закарпатський. Найбільший розрив між нашими моделями і комерційними базовими лініями: umt5 (27.45), MamaLM (20.93) проти Gemini (68.16), GPT-4o (65.39), Mistral (63.65). Як і для бойківського, корпус закарпатського сформований виключно з LLM-синтетики. Нуль-шотовий MamaLM (27.90) навіть злегка перевершує донавчений на синтетиці umt5 (27.45), що підтверджує: навчання на неякісній синтетиці може не давати жодного приросту або навіть шкодити.

Загальний висновок: спеціалізоване донавчання переконливо виправдане для різновидів із якісними навчальними даними (суржик, гуцульський). Для бойківського і закарпатського, де корпус складається виключно з LLM-синтетики, комерційні ВММ у нуль-шотовому режимі залишаються сильнішими. Ключовий фактор якості - не обсяг синтетики, а наявність ручних анотацій.

3.5 Вебзастосунок «Лексикон»

3.5.1 Архітектура застосунку

Застосунок «Лексикон» реалізовано на базі фреймворку NiceGUI версії 1.4 і вище. NiceGUI будує Python-native UI над FastAPI і Starlette: розробник описує інтерфейс через Python-об'єкти (ui.button, ui.textarea тощо), а фреймворк генерує відповідний HTML/JavaScript і обслуговує його через вбудований ASGI-сервер. Перевага такого підходу - відсутність необхідності писати JavaScript вручну: весь код інтерфейсу, моделі та обробки запитів знаходиться в одному Python-середовищі.

Застосунок запускається командою make demo (або безпосередньо uv run python app.py) і доступний за адресою <http://0.0.0.0:7870>. При старті завантажуються обидва модулі: нормалізатор і модуль АРМ. Якщо ГП недоступний,

нормалізатор автоматично перемикається на ЦП-режим (float32 замість float16), що дозволяє запустити застосунок навіть без відеокарти з певним зниженням швидкості.

Інтерфейс застосунку показано на рис. 3.10.

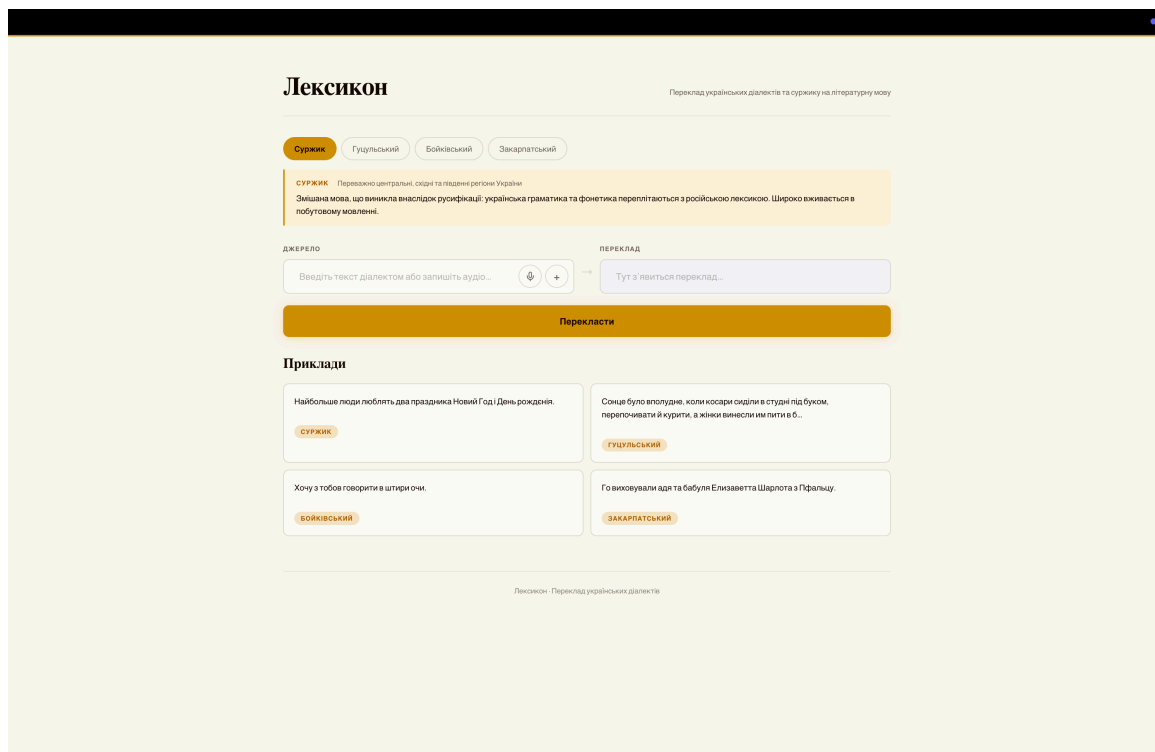


Рис. 3.10: Інтерфейс вебзастосунку «Лексикон»: вибір різновиду мови, поле введення тексту з кнопкою мікрофону та поле нормалізованого результату.

3.5.2 Модуль нормалізації

Клас DialectTranslator завантажує збережену мультідіалектну модель umt5-base з каталогу models/umt5-base-multidialect/final_model у режимі float16 на ГП. Обумовлення на конкретний діалект реалізується через префікс запиту:

- «surzhyk» → «Переклади з суржику: {текст}»;
- «hutsul» → «Переклади з гуцульської: {текст}»;
- «boiko» → «Переклади з бойківської: {текст}»;
- «transcarpathian» → «Переклади з закарпатської: {текст}».

Вхідний текст токенізується і передається на генерацію через model.generate() з такими параметрами: num_beams=5 (пошук з 5 променями), max_length=128, repetition_penalty=1.2, no_repeat_ngram_size=3.

Параметр `repetition_penalty=1.2` знижує ймовірність повторення слів у виводі, а `no_repeat_ngram_size=3` забороняє повтор будь-якого тримграму - разом це усуває артефакти зациклення, характерні для seq2seq-моделей на коротких або незнайомих входах.

Користувач може динамічно регулювати `num_beams` (від 3 до 10) та `repetition_penalty` (від 1.0 до 2.0) через повзунки в інтерфейсі без перезавантаження моделі. Зміна `num_beams` впливає на якість і швидкість: більше променів дає точніший результат, але збільшує час відповіді.

3.5.3 Модуль автоматичного розпізнавання мовлення

Модуль APM використовує модель `speechbrain/m-ctc-t-large` (MSTCTForCTC через Hugging Face Transformers) - багатомовну CTC-модель, попередньо навчену на 7 мовах без додаткового донавчання на діалектних даних. Модель обрана через підтримку кириличного алфавіту і відносно невеликий розмір (близько 450 МБ) для моделі такого класу.

Обробка аудіо реалізована через бібліотеку PyAV: браузер записує мовлення через MediaRecorder API і надсилає файл у форматі WebM або OGG на ендпоінт FastAPI POST `/api/transcribe`. На сервері PyAV декодує аудіопотік, конвертує у моно float32 і ресемплює до 16 кГц - стандартна частота дискретизації для більшості APM-моделей. Отриманий масив передається у модель, результат декодується через AutoProcessor і повертається у відповіді як JSON: `{"text": "..."}.`

Перевага використання PyAV замість зовнішнього ffmpeg: бібліотека встановлюється через pip і містить всі необхідні кодеки статично, що спрощує розгортання на нових машинах без системних залежностей.

3.5.4 Інтерфейс користувача

Інтерфейс організований у три колонки на десктопі: ліва - панель вибору діалекту з прикладами, центральна - поле введення тексту, права - поле результату. На мобільних пристроях (ширина менш ніж 768 пікселів) колонки перешкоджаються у вертикальний стек через CSS breakpoint.

Функціонал застосунку:

- чотири кнопки вибору діалекту з коротким описом і географічним регіоном;
- поле введення тексту з кнопкою мікрофону для голосового введення через АРМ - натискання починає запис, повторне натискання зупиняє і автоматично транскрибує мовлення у поле введення;
- кнопка «Нормалізувати» запускає обробку і виводить результат у правому полі;
- повзунки для тонкого налаштування параметрів генерації (num_beams, repetition_penalty);
- чотири приклади вхідних речень (по одному на кожен різновид) для швидкого тестування без введення тексту вручну.

Весь інтерфейс описано приблизно у 250 рядках Python-коду у файлі `app.py`, що є наслідком декларативного підходу NiceGUI.

3.6 Висновки до розділу

У розділі описано програмне та технічне забезпечення системи «Лексикон». Технологічний стек базується на Python 3.11 з бібліотеками PyTorch 2.6.0, Transformers, PEFT, TRL та Unsloth; навчання відбувалося на NVIDIA A100 (80 ГБ VRAM). Конвеєр генерації корпусу реалізує два підходи: правилний через клас `SurzhykErrorifier` (20 911 прикладів суржику з `rumorphy2` і словника) та LLM-генерація через клас `DialectCorpusGenerator` (108 812 прикладів через `Mistral-Small-24B-Instruct-2501`). Загалом корпус налічує 125 615 навчальних пар.

Навчання `umt5-base` виконано через `Seq2SeqTrainer` за 40 епох з раннім зупинком (терпіння 5 епох), навчання `MamayLM` - через `SFTTrainer` з QLoRA та пакуванням за 3 епохи. Оцінювання на 200 ручних прикладах показало, що донавчені моделі перевершують усі нуль-шотові базові лінії на суржику (`chrF++` = 93.58 для `umt5-base` проти 86.67 у `Gemini 2.0 Flash`) та гуцульському (86.90 проти 65.34). Для бойківського і закарпатського, де навчальний корпус склада-

ється виключно з LLM-синтетики, комерційні ВММ у нуль-шотовому режимі виявились кращими (Gemini: 74.57 і 68.16 відповідно), що вказує на критичну роль якісних ручних анотацій. Вебзастосунок реалізовано на NiceGUI і FastAPI з нормалізатором на основі umt5-base і АРМ-модулем speechbrain/m-ctc-t-large.

ЗАГАЛЬНІ ВИСНОВКИ

У роботі розроблено систему «Лексикон» для автоматичної нормалізації суржику та трьох регіональних діалектів сучасної української мови до літературної норми.

Основні результати роботи:

1. Проаналізовано сучасні підходи до нормалізації діалектів і суржику та найближчі роботи у цій галузі (UA-GEC, Spivavtor, Vuuko Mistral, Bytes to Borsch). Встановлено відсутність публічних паралельних корпусів для бойківського та закарпатського діалектів.
2. Зібрано та проаналізовано вхідні дані: ручний паралельний корпус гуцульського діалекту (9 853 пари), загальномовний корпус Spivavtor [12.] як джерело нормативних речень. Проаналізовано обмеженість наявних ресурсів і обґрунтовано підхід до синтетичного розширення.
3. Розроблено конвеєр правилкової генерації суржику (клас `SurzhykErrorifier`): двофазовий алгоритм на основі словника і морфологічного аналізатора `rumorphy2` дав 20 911 навчальних прикладів.
4. Розроблено конвеєр LLM-генерації діалектних пар (клас `DialectCorpusGenerator`) з відбором словникових статей через TF-IDF косинусну схожість. Згенеровано 108 812 прикладів через `Mistral-Small-24B-Instruct-2501`. Підсумковий корпус: 125 615 навчальних / 13 756 валідаційних / 200 тестових пар.
5. Виконано повне донавчання кодер-декодерної моделі `umt5-base` [3.] (40 епох, $lr = 5 \times 10^{-5}$, згладжування міток $\alpha = 0.1$, рання зупинка). Результат на тестовій вибірці: `chrF++` = 93.58 на суржику, 86.90 на гуцульському.
6. Виконано QLoRA-донавчання декодерної BMM `MamayLM` [11.] (`Gemma-3-4B`, $r = 16$, $\alpha = 32$, 3 епохи, прискорення `Unsloth`). Резуль-

тат: chrF++ = 91.22 на суржику, 66.82 на бойківському.

7. Реалізовано вебзастосунок «Лексикон» на NiceGUI і FastAPI з нормалізатором umt5-base (пошук з 5 променями) і модулем автоматичного розпізнавання мовлення speechbrain/m-ctc-t-large для голосового введення.

Основний науковий результат: спеціалізоване донавчання перевершує нуль-шотові комерційні ВММ (GPT-4o, Gemini 2.0 Flash, Mistral Large) для різновидів із якісними навчальними даними - суржику (chrF++ = 93.58 проти 86.67 у Gemini) та гуцульського (86.90 проти 65.34). Для бойківського і закарпатського, де навчальний корпус складається виключно з LLM-синтетики, перевага залишається за комерційними ВММ (Gemini: 74.57 і 68.16 відповідно). Це підтверджує ключову роль ручних анотацій: не обсяг синтетики, а її якість і наявність ручних еталонних пар визначають здатність моделі перевершити загальні ВММ.

Напрями подальшого розвитку: зібрати ручні паралельні корпуси для бойківського (мінімум 3-5 тис. пар) і закарпатського діалектів - це є головним вузлом місцем системи; донавчити модуль АРМ на діалектних аудіозаписах; розширити підтримку інших діалектів (слобожанський, подільський, волинський); розробити програмний інтерфейс для інтеграції «Лексикону» з іншими інструментами обробки природної мови.

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

1. Baniata L. H., Park S., Park S.-B. A Neural Machine Translation Model for Arabic Dialects That Utilizes Multitask Learning (MTL) // *Computational Intelligence and Neuroscience*. 2018. С. 1-10. doi: 10.1155/2018/7534712.
2. Bondarenko M., Yushko A., Shportko A., Fedorych A. Comparative Study of Models Trained on Synthetic Data for Ukrainian Grammatical Error Correction // *Proceedings of the Third Ukrainian Natural Language Processing Workshop (UNLP)*. 2024.
3. Chung H. W. et al. UniMax: Fairer and more Effective Language Sampling for Large-Scale Multilingual Pretraining. Квітень 2023. arXiv:2304.09151. doi: 10.48550/arXiv.2304.09151.
4. Fedchuk R., Vysotska V. Mathematical Model of a Decision Support System for Identification and Correction of Errors in Ukrainian Texts Based on Machine Learning // *CEUR Workshop Proceedings*. 2021.
5. Haltiuk M. On the Path to Make Ukrainian a High-Resource Language // *Proceedings of the Second Ukrainian Natural Language Processing Workshop (UNLP)*. 2023.
6. Kent K. Language Contact: Morphosyntactic Analysis of Surzhyk Spoken in Central Ukraine // *Journal of Language Contact*. 2010.
7. Kiulian A. et al. From Bytes to Borsch: Fine-Tuning Gemma and Mistral for the Ukrainian Language Representation. Квітень 2024. arXiv:2404.09138. doi: 10.48550/arXiv.2404.09138.
8. Kovalchuk R., Romanyshyn M., Ivaniuk P. Introducing OmniGEC: A Silver Multilingual Dataset for Grammatical Error Correction // *Proceedings of the Second Ukrainian Natural Language Processing Workshop (UNLP)*. 2023.
9. Kyslyi R., Maksymiuk Y., Pysmennyi I. Vuyko Mistral: Adapting LLMs for Low-Resource Dialectal Translation. Червень 2025. arXiv:2506.07617. doi: 10.48550/arXiv.2506.07617.
10. Lapa LLM.

Доступно у: <https://huggingface.co/lapa-llm>

11. MamayLM.

Доступно у: <https://huggingface.co/blog/INSAIT-Institute/mamaylm>

12. Saini A., Chernodub A., Raheja V., Kulkarni V. Spivavtor: An Instruction Tuned Ukrainian Text Editing Model. Квітень 2024. arXiv:2404.18880. doi: 10.48550/arXiv.2404.18880.

13. Sira N., Di Nunzio G. M., Nosilia V. Towards an Automatic Recognition of Mixed Languages in R: The Case of Ukrainian-Russian Hybrid Language Surzhyk // *CLiC-it 2021*. 2021.

14. Syvokon O., Nahorna O., Kuchmiichuk P., Osidach N. UA-GEC: Grammatical Error Correction and Fluency Corpus for the Ukrainian Language // *Proceedings of the Second Ukrainian Natural Language Processing Workshop (UNLP)*. 2023. С. 96-102. doi: 10.18653/v1/2023.unlp-1.12.

15. Xue L. et al. ByT5: Towards a Token-Free Future with Pre-Trained Byte-to-Byte Models // *Transactions of the Association for Computational Linguistics*. 2022. arXiv:2105.13626. doi: 10.48550/arXiv.2105.13626.

16. Zhao H. et al. From Handcrafted Features to LLMs: A Brief Survey for Machine Translation Quality Estimation // *IJCNN 2024*. Чер 2024. С. 1-10. doi: 10.1109/IJCNN60899.2024.10650457.

Додаток А

Повний вихідний код системи «Лексикон» - скрипти генерації та підготовки корпусу, конфігурації донавчання моделей `umt5-base` і `MamayLM`, код оцінювання та вебзастосунку, а також повні тексти системних промптів для генерації суржикових і діалектних (гуцульських, бойківських, закарпатських) пар через модель `Mistral-Small-24B-Instruct-2501` - розміщено у відкритому репозиторії на платформі GitHub за адресою: <https://github.com/DenkoProg/surdo-perevodchik>.

Репозиторій містить теку `prompts/` із системними промптами для кожного мовного різновиду (лінгвістичні правила, словники підстановок і приклади перекладів), теку зі скриптами формування корпусу, файли конфігурації навчання та код застосунку з модулями нормалізації й автоматичного розпізнавання мовлення. Усі результати роботи відтворюються командами, описаними у файлі `Makefile` репозиторію.